Efficient models: GPU

Egor Shvetsov

1. **Motivation & Efficiency Metrics** FLOPs, MACs, Latency, Energy

2. **Data Types** FP32, FP16, BF16, INT8, FP8 and more

3. **GPU Architecture** SM, Warps, Blocks, Memory Hierarchy

4. **Arithmetic: Time & Energy Cost** MatMul cost analysis

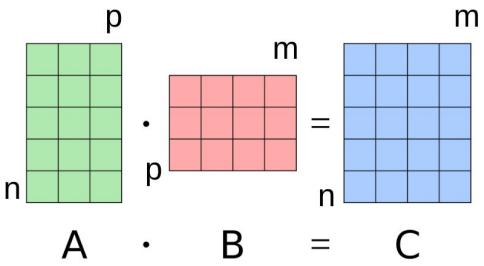
5. **GPU Execution Model** Tiling, CUDA code, Warps in practice

Memory
FLOPs
MAC
Latency
Energy

FLOPs - number of floating point operations?

How much is it for a matrix multiplication?

$$F[(n \times p) \times (p \times m)] = ?$$



1. Motivation & Efficiency Metrics FLOPs, MACs, Latency, Energy

Motivation: Counting FLOPs for Matrix Multiplication

For C = A × B where A is (n×p) and B is (p×m):

Each output element C[i,j] requires:

p multiplications + p additions = 2p FLOPs

Total output elements: n × m

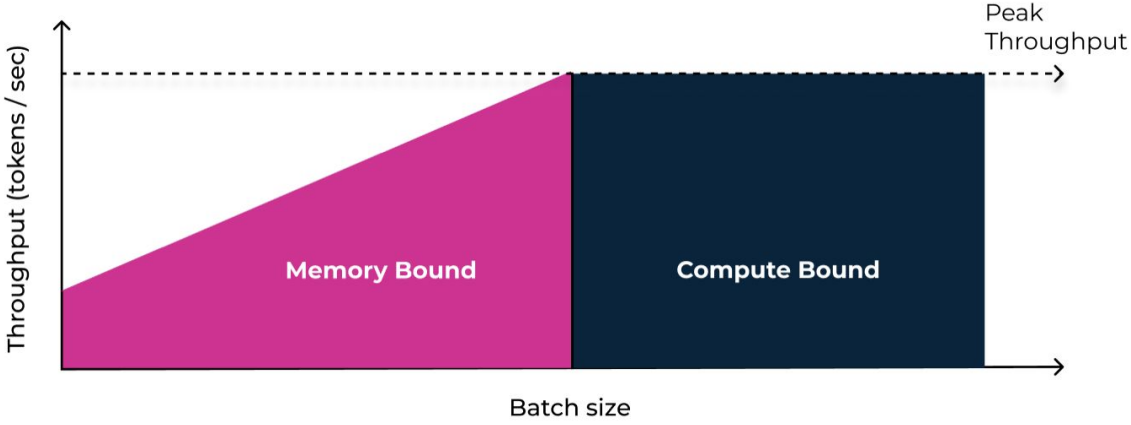
Total FLOPs = 2 × n × p × m

With FMA (Fused Multiply-Add): a * b + c atomically, without rounding the intermediate result.

Each multiply+add counts as 1 FMA = 1 FLOP

So effective FLOPs = n × p × m (MACs)

Memory
FLOPs
MAC
Latency
Energy



model	gpu	task	energy	throughput	response_length	latency	arc	hellaswag	truthfulqa	parameters
MetaAI/Llama-7B	A100	chat	335.26	27.47	60.65	1.99	51.11	77.74	34.08	7
MetaAI/Llama-13B	A100	chat	631.57	23.2	76.47	2.97	56.31	80.86	39.9	13
tatsu-lab/alpaca-7B	A100	chat	684.17	28	132.85	4.63	52.65	76.91	39.55	7
RWKV/rwkv-raven-7b	A100	chat	735.77	61.52	214.78	3.08	39.42	66.45	38.54	7
Neutralzz/BiLLa-7B-SFT	A100	chat	881.44	33.57	161.81	4.79	27.73	26.04	49.05	7
H2OAI/H2OGPT-oasst1-7B	A100	chat	951.08	33.2	212.46	6.39	36.86	61.55	37.94	7
FreedomIntelligence/phoenix-inst-chat-7b	A100	chat	1030.89	55.84	240.34	4.12	44.97	63.22	47.08	7
StabilityAI/stablelm-tuned-alpha-7b	A100	chat	1124.4	45.42	243.88	5.21	31.91	53.59	40.22	7
databricks/dolly-v2-12B	A100	chat	1242.92	22.11	153.76	7.94	42.15	71.83	33.37	12
Salesforce/xgen-7b-8k-inst	A100	chat	1246.09	49.54	275.27	5.6	46.67	74.85	41.89	7
BAIR/koala-7b	A100	chat	1345.55	26.56	261.07	9.62	47.1	73.7	46	7
togethercomputer/RedPajama-INCITE-7B-Chat	A100	chat	1457.86	27.49	274.39	9.54	42.15	70.84	36.1	7
LMSys/vicuna-7B	A100	chat	1531.7	27.26	284.92	10.3	53.5	77.53	49	7
project-baize/baize-v2-7B	A100	chat	1588.27	31.59	326.3	10.17	48.46	75	41.66	7
LMSys/fastchat-t5-3b-v1.0	A100	chat	1802.98	19.29	314.3	20.68	35.92	46.36	48.79	3
nomic-ai/gpt4all-13b-snoozy	A100	chat	2004.73	26.57	247.8	9.28	56.06	78.69	48.36	13
OpenAssistant/oasst-sft-1-pythia-12b	A100	chat	2060.03	25.4	259.59	9.93	45.56	69.93	39.19	12
BAIR/koala-13b	A100	chat	2144.83	21.21	265.57	12.14	52.9	77.54	50.09	13
openaccess-ai-collective/manticore-13b-cl	A100	chat	2189.25	26.28	288.82	10.95	58.7	81.96	48.86	13
Camel-AI/CAMEL-13B-Combined-Data	A100	chat	2526.46	24.5	291.92	13.17	55.55	79.3	47.33	13
LMSys/vicuna-13B	A100	chat	2600.84	21.61	280.74	12.74	52.9	80.12	51.82	13

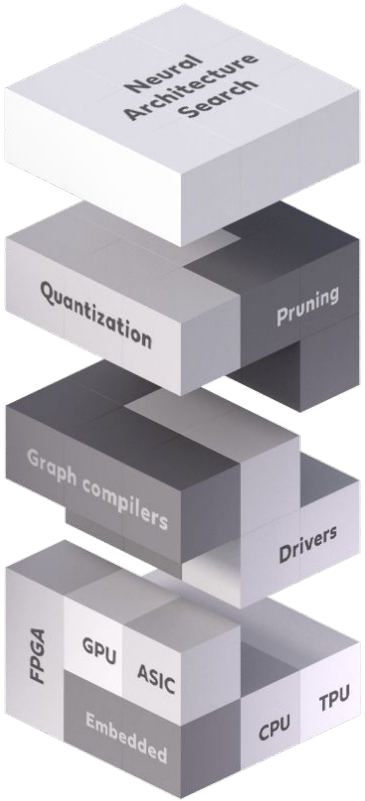
How much energy consumes LLama-7B in terms of 60W a light bulb:

- 3 seconds of work
- 5 seconds of work
- 60 seconds on work

Brain consumes 20 joules energy per second.

source: ml.energy

2. Data Types FP32, FP16, BF16, INT8, FP8 and more



HARDWARE

- GPU
- TPU
- CPU
- FPGA
- Ascend
- etc.

SOFTWARE

- CUDA
- Graph Compilers
- MKL - DNN
- etc.

ALGORITHMIC

- PRUNING
- QUANTIZATION
- NAS
- etc.

A majority of machine learning architectures lack explicit consideration for software or hardware properties during development, resulting in computational inefficiency.

Furthermore, many models concentrate exclusively on achieving optimal results rather than accounting for practical implementation issues.

Image source: deci.ai

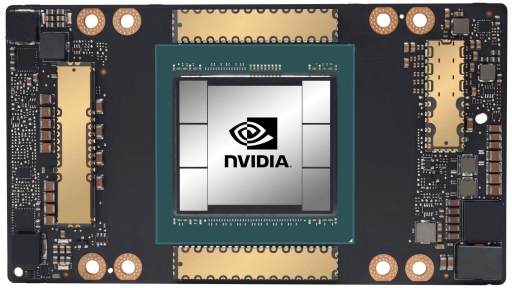
Support of low precision computations in GPU

- Float16
- Float32
- Bfloat16

Will be discussed in future series …

Not supported by hardware or software

32 bit | 16 bit | 8 bit | 4 bit | 1 bit

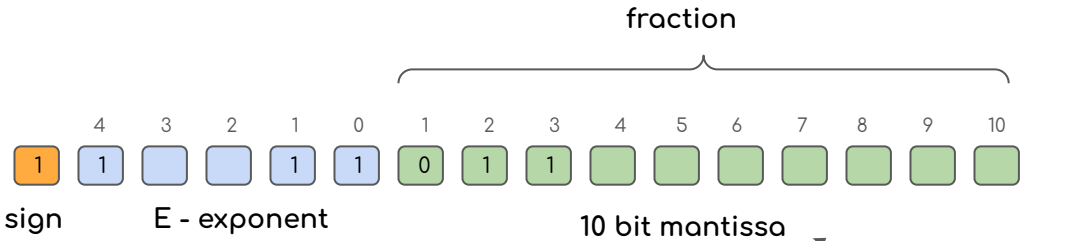


Supported by some hardware only

Int16/Int8/Int4

Range of values: $[-2^{n-1}, 2^{n-1} - 1]$

Float32 and Float16
(Floating Point)



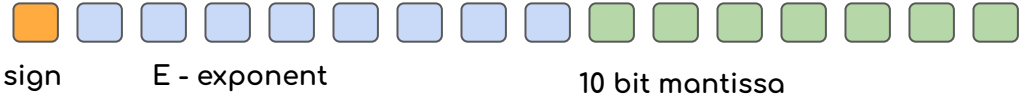
$$\mathbf{v} = (-1)^1(1+f)2^{e-bias} \quad bias = 15$$

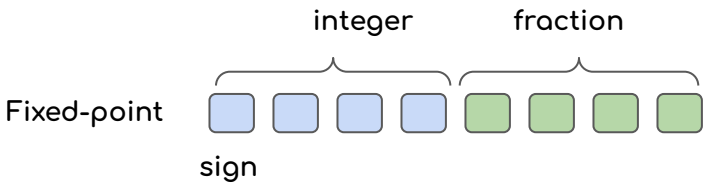
$$f = 2^{-2} + 2^{-3} = \frac{1}{4} + \frac{1}{8} = 0.375$$

$$e = 2^0 + 2^1 + 2^4 = 19$$

$$\mathbf{v} = (-1)^1(1+0.375)2^{19-15} = 22$$

Bfloat16





MORE:

TensorFloat-32 19 bit (supported in A100) : 1 bit sign, 8 bit exponent, 10 bit —mantissa.

E4M3 (8bit) 1 bit sign, 4 bit exponent, 3 bit mantissa
E5M2 (8bit) 1 bit sign, 5 bit exponent, 2 bit mantissa
E2M1 (4bit) 1 bit sign, 2 bit exponent, 1 bit mantissa
E3M0 (4bit) 1 bit sign, 3 bit exponent

So, Float8 and int8 or Float4 and int4 use the same number of bits.
Why do we care about int4 or int8 then?

E4M3 (8bit) 1 bit sign, 4 bit exponent, 3 bit mantissa
E5M2 (8bit) 1 bit sign, 5 bit exponent, 2 bit mantissa
E2M1 (4bit) 1 bit sign, 2 bit exponent, 1 bit mantissa
E3M0 (4bit) 1 bit sign, 3 bit exponent

Int16/Int8/Int4

sign 3 bits

Range of values: $[-2^{n-1}, 2^{n-1} - 1]$

Support of low precision computations in GPUs / CPUs

Data Type	NVIDIA GPUs	AMD GPUs	Intel CPUs	Google TPU	AWS Inferentia/Trainium
FP32	"OK" All architectures	"OK" All CDNA/RDNA	"OK" AVX-512	"OK" (training fallback)	"OK" NeuronCore v1/v2
FP16	"OK" Tensor Cores (Volta+)	"OK" Matrix Cores (CDNA1+)	AVX-512 (software)	Limited support	"OK" NeuronCore v1/v2
BF16	"OK" Ampere+ (A100, H100) 5	"OK" CDNA1+ (MI100+) 11	"OK" AMX (4th Gen Xeon) 22	"OK" Primary training format	"OK" NeuronCore v1/v2
TF32	"OK" Ampere+ (A100, H100)	"NO"	"NO"	Internal use	"OK" NeuronCore v2 41
FP8 (E4M3/E5M2)	"OK" Hopper (H100/H200) + Transformer Engine 13	"OK" CDNA3/CDNA4 (MI300+) FNUZ/OCP variants 11	"NO"	Research/experimental	"OK" cFP8 NeuronCore v2 41
FP6/FP4	"OK" FP4 weight-only quantization (TensorRT) 4	"OK" FP4/FP6 on CDNA4 (MI350X+) 11	"NO"	"NO"	"NO"
INT8	"OK" Tensor Cores (all recent) + sparsity	"OK" Matrix Cores + 2:4 sparsity 70	"OK" AMX (4th Gen Xeon) 22	"OK" Inference optimized	"OK" NeuronCore v2 (UINT8) 45
INT4/UINT4	"OK" Weight-only quantization (TensorRT) 4	Experimental	"NO"	"NO"	"NO"
UINT8	"OK" I/O only (TensorRT) 4	"OK"	"NO"	Limited	"OK" NeuronCore v2 45
BOOL	"OK" (TensorRT) 4	"OK"	"OK"	"OK"	"OK"

Support of low precision computations in ASCEND

Data Type	Format Details	Training Support	Inference Support	Hardware Availability	Notes
HiF8 (HiFloat8)	Tapered precision: dynamic exponent/mantissa allocation (7E/3M, 8E/2M, 16E/1M modes) + extended denormals	"OK" Yes (forward + backward passes) 11	"OK" Yes	Ascend 910B+, CANN 7.0+	Huawei's proprietary alternative to OCP FP8; better dynamic range than E4M3/E5M2 15
HiF4 (HiFloat4)	4-bit with 3-level hierarchical scaling	Experimental	"OK" Yes (PTQ-optimized) 20	Ascend 910B+, A5 series	Designed for weight-only KV-cache quantization; avoids accuracy collapse of INT4 20
MXFP8 (Microscaling)	Block-scaling format: 32-element blocks with shared E8M0 scale + E4M3/E5M2 elements	Framework-dependent	"OK" Yes (via CANN plugins)	Ascend A5+ (2026)	Support in development: vLLM-Ascend RFC targets Q1 2026 for MXFP8-W8A8 dense/sparse models 2324
FP4 (weight-only)	Ultra-low precision weight quantization	No	"OK" Yes (Atlas 950/960)	Ascend 950PR/DT (late 2026)	Enables 26.5× inference TPS gain on Atlas 950 supernode 5
INT8/INT4	Standard integer quantization	QAT required	"OK" Yes	All Ascend NPUs	Native Da Vinci core support; sparse INT8 via hipSPARSELt-equivalent ops
Structured Sparsity (2:4)	Weight pruning pattern	"OK" With pruning tools	"OK" Optimized kernels	Ascend 910B+	Requires model preparation; supported via CANN sparse operators 40

Format	Public Documentation	Hardware Support	Software Stack	Production Ready?
HiF8	[V] White paper (arXiv:2409.16626)	[V] Ascend 910B+	[V] CANN 7.0+, MindSpore 2.3+	[V] Yes (training + inference)
HiF4	[V] Evaluation paper (arXiv:2602.12635)	[V] Ascend 910B+, A5	[V] CANN 7.5+ (PTQ tools)	[V] Yes (inference only)
MXFP8	[~] RFC on GitHub	[V] Ascend A5 (2026)	[~] CANN 8.0 (Q1 2026 target)	[~] Early access / preview
FP4	[~] Roadmap blogs	[V] Ascend 950/960 (late 2026)	[~] CANN 8.0+	[X] Not yet commercially available
Sparse + Low-Precision	[~] Limited docs	[V] 910B+ (2:4)	[V] CANN sparse ops	[V] Yes (with model prep)

Sparsity data types

Hardware	Sparse Format	Pattern	Training	Inference	Notes
NVIDIA A100/H100	2:4 structured sparsity	Weight matrices	"OK" (with pruning)	"OK" Optimized	Requires 2 non-zero values per 4-element block 5358
AMD MI300 Series	2:4 structured sparsity	INT8/FP8/FP16/BF16	"OK" (hipSPARSELt)	"OK" 2x perf gain	Native hardware support via Matrix Cores 7074
Intel 4th Gen Xeon	Unstructured sparsity	Embedding/AMX ops	Software-assisted	"OK" Optimized	Via oneMKL Sparse BLAS; limited native acceleration 7981
Google TPU v4+	SparseCore	Embedding tables	"OK" Recommendation models	"OK" High-throughput	Dedicated dataflow processors for sparse/dense mixed workloads 6567
AWS Trainium/Inferentia	Limited public docs	—	Framework-dependent	"OK" Via Neuron SDK	Sparsity handled at framework level (PyTorch/TensorFlow)

Binary data types

Vendor	Implementation Method	Performance Gain	Software Stack
NVIDIA	Bit-packing into INT32 registers; custom CUDA kernels	2–3× vs. INT8 (theoretical 32×)	TensorRT (experimental plugins), Custom CUDA
AMD	ROCm libraries pack bits into INT8 wavefronts	2–3× vs. INT8	MIOpen, Custom HIP kernels
Intel	AVX-512 VNNI + Bit manipulation instructions	3–4× on Xeon (CPU-bound)	oneDNN, OpenVINO (BNN support)
Google	TPU INT8 units simulate binary via masking	Minimal gain vs. INT8	JAX/TensorFlow quantization aware training
AWS	Neuron SDK maps binary ops to INT8 hardware	Minimal gain vs. INT8	Neuron SDK (Quantization tools)
Huawei	CANN custom operators for bit-packing	2–3× vs. INT8	MindSpore (Binary Connect operators), CANN

Feature	Status on Modern CPUs (Intel/AMD)	Explanation
Bitwise Instructions (AND, XOR)	[V] Native	Single-cycle instructions available on all cores.
Population Count (POPCNT)	[V] Native	Hardware instruction to count "1" bits in a register.
Vectorized Bitwise (AVX-512)	[V] Native	Can process 512 bits of binary data in parallel per clock.
Dedicated Matrix Unit (AMX)	[X] No Support for Binary	Intel AMX tiles only accept INT8, BF16, FP16.
Vector Neural Network (VNNI)	[X] No Support for Binary	Intel/AMD VNNI instructions optimize INT8 multiply-accumulate, not binary.
Binary Matrix Multiply (GEMM)	[~] Software-Constructed	Built by combining AVX-512 + POPCNT manually (via libraries like oneDNN).

Binary operations on CPU:

Basic Operations (XOR, POPCouNT):
[V] Yes, Native. CPUs have hardware instructions for bitwise math and counting bits.

Matrix Multiplication Engine (AMX/VNNI):
[X] No, Not for Binary.

Result:
We can use binary matrix multiply using vector instructions, but it does not get the dedicated acceleration that INT8 gets.

Binary data types

So, do we need binary weights at all if currently there is no way to accelerate math?

Pros and cons?

Binary data types

Cons: Slower math, lower accuracy and bit packing overhead

Pros: Memory and energy savings

And perhaps, in the future we will have better math support ...

3. GPU Architecture SM, Warps, Blocks, Memory Hierarchy

Throughput optimized

Latency optimized

SIMD - Single Instruction Multiple Data

StMp - Single Truck Multiple Products

132

Streaming Multiprocessors

in H100

8,448

CUDA Cores (FP32)

64 per SM × 132 SM

80 GB

HBM3 Memory

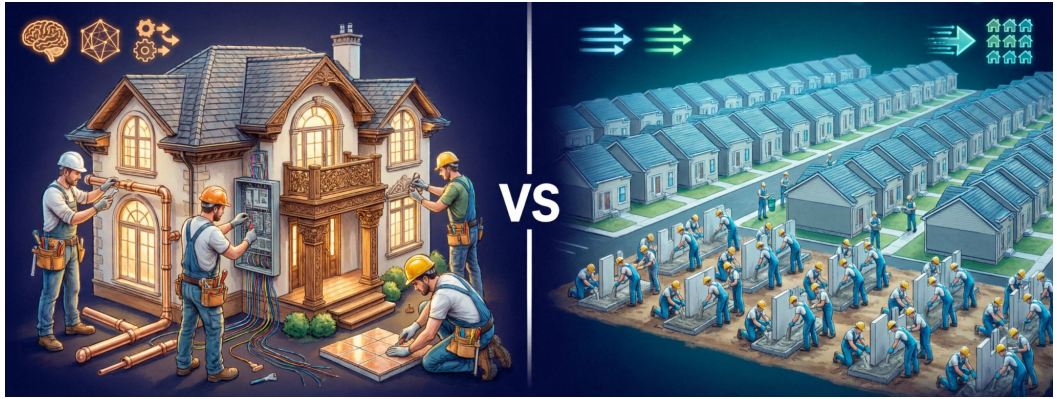
3.35 TB/s bandwidth

GPU Architecture Complexity:

The Hopper architecture whitepaper is 71 pages. The PTX ISA reference is 598 pages.
80 billion transistors in H100. No one knows every detail.

Key insight: GPU trades latency for throughput.

CPU optimizes for sequential speed (few powerful cores).
GPU maximizes parallel throughput (thousands of simpler cores).



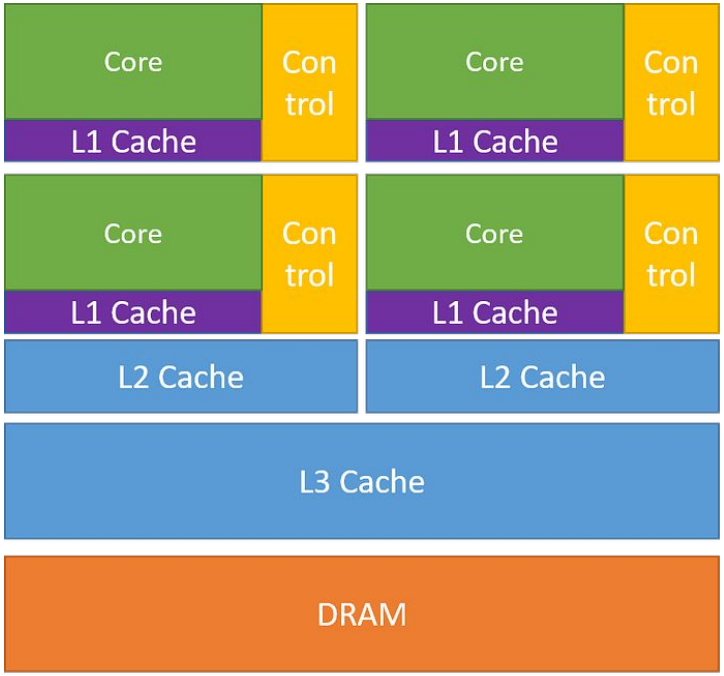
Imagine, you have a thousand of workers and you need to balance efficiency / flexibility in your work.

Then imagine that you office space is limited and you can assign only a fixed number of people to seat there.

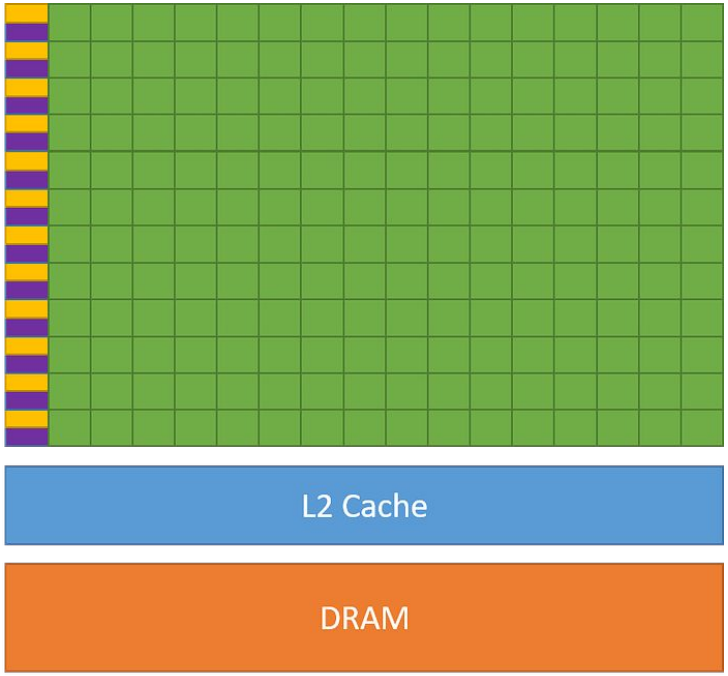
Adding more managers and shared resources would make you more flexible but it will take some resources.

What to do?

Архитектура GPU



CPU

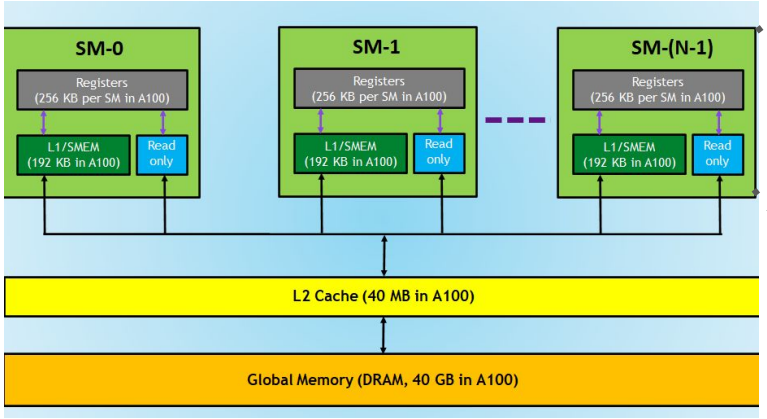


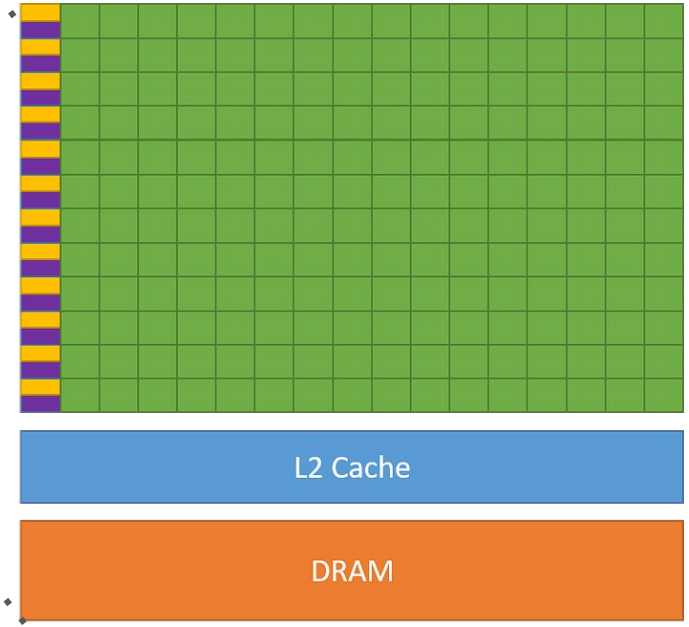
GPU

CPU: большие кэши и большое количество блоков управления.

GPU: большое количество ALU для максимизации своей вычислительной мощности и пропускной способности. Мало кэшей и блоков управления.

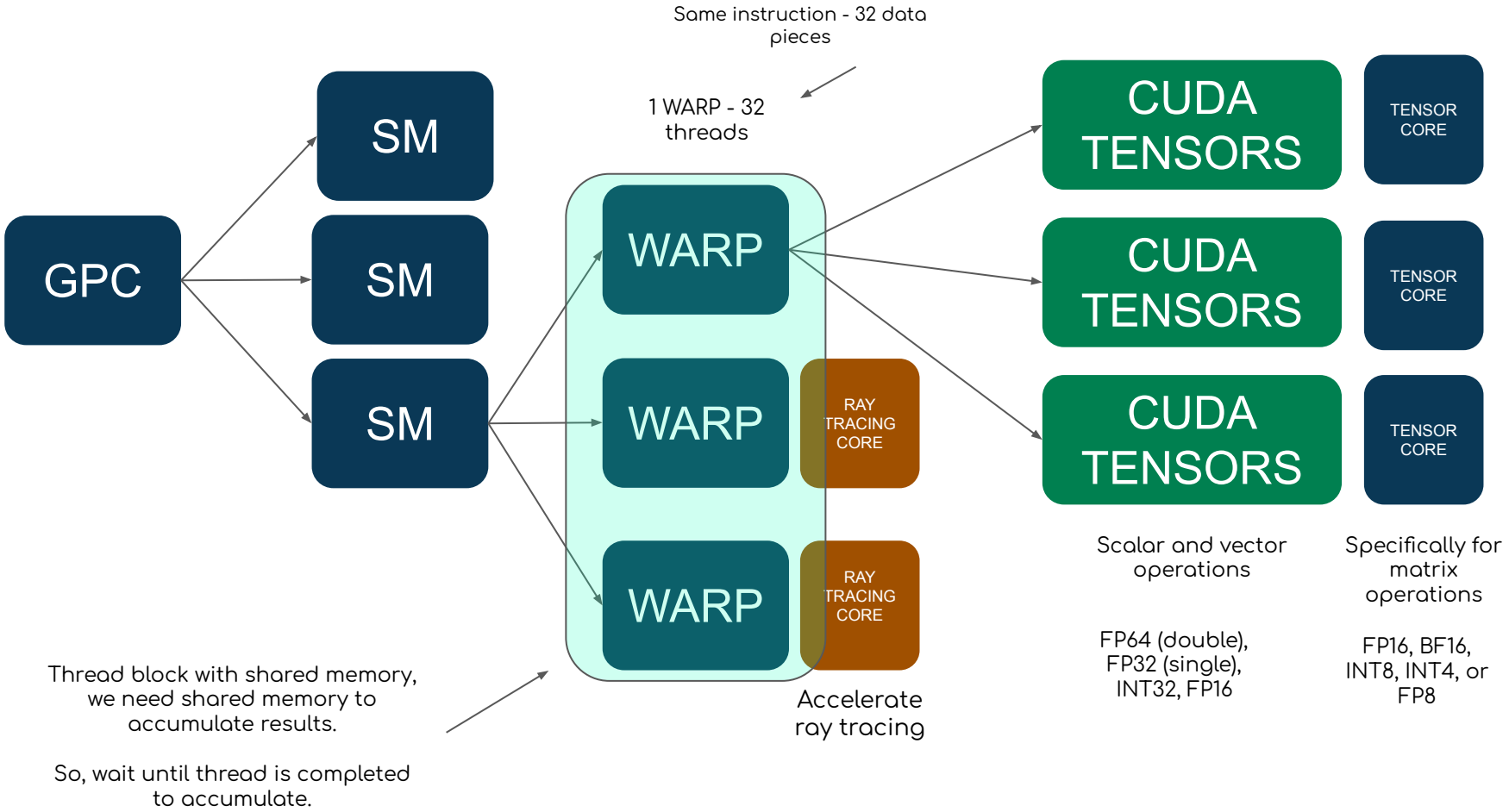
Архитектура GPU

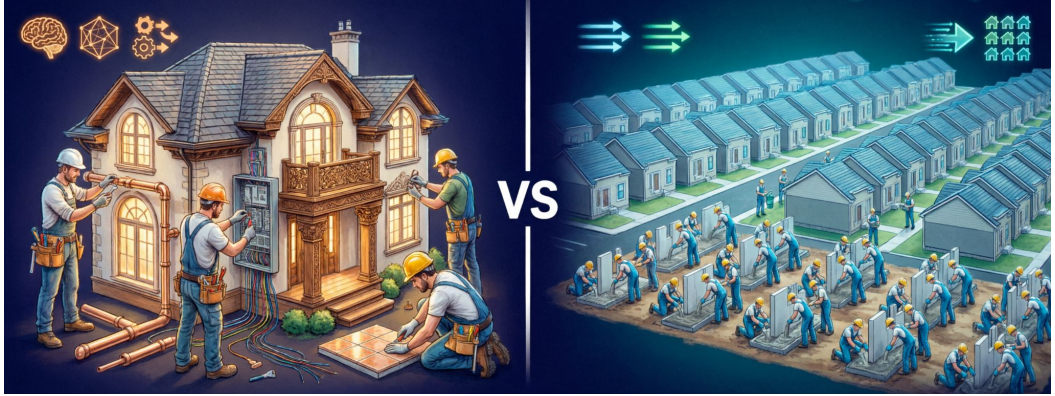




GPU

- GPU состоит из массива потоковых мультипроцессоров - Streaming Multiprocessor (SM).
- Каждый SM, в свою очередь, включает несколько потоковых процессоров, также называемых ядрами или потоками. Например, GPU Nvidia H100 содержит 132 SM с 64 ядрами на каждый SM, что в сумме даёт 8448 ядер.
- Каждый SM имеет ограниченное количество встроенной памяти, часто называемой общей памятью, которая используется всеми ядрами совместно.
- Ресурсы блока управления на SM разделяются между всеми ядрами.
- Каждый SM оснащен аппаратными планировщиками потоков для выполнения потоков (тредов).
- Помимо этого, каждый SM содержит несколько функциональных блоков или других ускоренных вычислительных устройств.


Same instruction - 32 data pieces
1 WARP - 32 threads
GPC
SM
SM
SM
WARP
WARP
WARP
RAY TRACING CORE
RAY TRACING CORE
CUDA TENSORS
CUDA TENSORS
CUDA TENSORS
TENSOR CORE
TENSOR CORE
TENSOR CORE
Scalar and vector operations
FP64 (double), FP32 (single), INT32, FP16
Specifically for matrix operations
FP16, BF16, INT8, INT4, or FP8
Accelerate ray tracing
Thread block with shared memory, we need shared memory to accumulate results.
So, wait until thread is completed to accumulate.



Концепция строительной компании	Компонент GPU	Объяснение
Вся строительная компания	GPU	Управляет тысячами рабочих на множестве объектов
Региональное подразделение	GPC (Graphics Processing Cluster)	Группы строительных объектов, совместно использующие базы техники
Строительный объект	SM (Streaming Multiprocessor)	Один объект со своим прорабом, оборудованием и бригадой
Проект жилого комплекса	Block (Блок)	«Построить 128 одинаковых домов» — назначается одному объекту
Специализированная бригада (32 рабочих)	Warp (Варп)	Бригада каркасников, кровельщиков — все выполняют одну и ту же задачу
Отдельный рабочий	Thread (Поток)	Один рабочий в бригаде, делающий одну задачу
Личные инструменты	Registers (Регистры)	Личный молоток, дрель — мгновенный доступ
Общие инструменты бригады	Shared Memory (Разделяемая память)	Гвоздомёты, лестницы — используются бригадой на объекте
Центральная база материалов	Global Memory (Глобальная память / VRAM)	Краска, кирпич, каркасные блоки — медленный доступ, используются всеми
Прораб на объекте	Scheduler (Планировщик)	Распределяет бригады по задачам, обеспечивает непрерывную работу

Представьте, что вы управляете строительной компанией. Ваш CPU — это мастер по индивидуальному строительству: 4 эксперта строят один роскошный особняк с уникальными решениями. Это сложно, гибко, но медленно.

Ваш GPU — это застройщик типового жилья: вы только что выиграли контракт на строительство 10 000 одинаковых домов. Вам не нужны мастера-универсалы — вам нужны тысячи рабочих, выполняющих одни и те же задачи:

- Бригады каркасников собирают каркасы всех 10 000 домов
- Бригады электриков прокладывают проводку во всех 10 000 домов
- Бригады кровельщиков накрывают крыши всех 10 000 домов

В каждой бригаде 32 рабочих выполняют одну и ту же операцию на разных домах. Это и есть варп.
На каждом объекте есть свой прораб и общие инструменты — это SM с разделяемой памятью.

Концепция строительной компании	Компонент GPU	Объяснение
Вся строительная компания	GPU	Управляет тысячами рабочих на множестве объектов
Региональное подразделение	GPC (Graphics Processing Cluster)	Группы строительных объектов, совместно использующие базы техники
Строительный объект	SM (Streaming Multiprocessor)	Один объект со своим прорабом, оборудованием и бригадой
Проект жилого комплекса	Block (Блок)	«Построить 128 одинаковых домов» — назначается одному объекту
Специализированная бригада (32 рабочих)	Warp (Варп)	Бригада каркасников, кровельщиков — все выполняют одну и ту же задачу
Отдельный рабочий	Thread (Поток)	Один рабочий в бригаде, делающий одну задачу
Личные инструменты	Registers (Регистры)	Личный молоток, дрель — мгновенный доступ
Общие инструменты бригады	Shared Memory (Разделяемая память)	Гвоздомёты, лестницы — используются бригадой на объекте
Центральная база материалов	Global Memory (Глобальная память / VRAM)	Краска, кирпич, каркасные блоки — медленный доступ, используются всеми
Прораб на объекте	Scheduler (Планировщик)	Распределяет бригады по задачам, обеспечивает непрерывную работу

Question	Answer	Analogy
What is the Thread?	One Worker	Worker #5
What is the Data?	One House	House #42
What is the Task?	The Action	Installing a window
What is the Warp?	The Crew	32 workers installing windows on 32 houses simultaneously

Thread = One Data Element (processed by the same code).
Warp = 32 Data Elements (processed by the same instruction simultaneously).

Концепция строительной компании	Компонент GPU	Объяснение
Вся строительная компания	GPU	Управляет тысячами рабочих на множестве объектов
Региональное подразделение	GPC (Graphics Processing Cluster)	Группы строительных объектов, совместно использующие базы техники
Строительный объект	SM (Streaming Multiprocessor)	Один объект со своим прорабом, оборудованием и бригадой
Проект жилого комплекса	Block (Блок)	«Построить 128 одинаковых домов» — назначается одному объекту
Специализированная бригада (32 рабочих)	Warp (Варп)	Бригада каркасников, кровельщиков — все выполняют одну и ту же задачу
Отдельный рабочий	Thread (Поток)	Один рабочий в бригаде, делающий одну задачу
Личные инструменты	Registers (Регистры)	Личный молоток, дрель — мгновенный доступ
Общие инструменты бригады	Shared Memory (Разделяемая память)	Гвоздомёты, лестницы — используются бригадой на объекте
Центральная база материалов	Global Memory (Глобальная память / VRAM)	Краска, кирпич, каркасные блоки — медленный доступ, используются всеми
Прораб на объекте	Scheduler (Планировщик)	Распределяет бригады по задачам, обеспечивает непрерывную работу

GPU ~132 SM (H100)

GPC — Graphics Processing Cluster Groups of SMs sharing resources

SM — Streaming Multiprocessor 64 CUDA cores + Tensor Cores + 96 KB shared memory

Warp 32 threads executing same instruction (SIMT)

Thread 1 CUDA core, private registers

Hardware (physical, exists in silicon):

- **SM** — a real chip die region with its own ALUs, schedulers, register file
- **Warp** — 32 threads that literally share one instruction block on the SM
- **Thread** — maps to one core (one FMA unit)
- **Registers** — physical memory on the SM die
- **Shared Memory / L1 / L2** — physical SRAM on the SM die
- **HBM3** — physical memory chips sitting next to the GPU die

GPU ~132 SM (H100)

GPC — Graphics Processing Cluster Groups of SMs sharing resources

SM — Streaming Multiprocessor 64 CUDA cores + Tensor Cores + 96 KB shared memory

Warp 32 threads executing same instruction (SIMT)

Thread 1 CUDA core, private registers

Programming abstraction:

- **GPC** — mostly a hardware grouping but invisible to the programmer
- **Block** — the key abstraction: you declare a block size, the hardware maps it to an SM
- **Grid** — purely logical: your array of blocks across the problem
- **Thread index** (threadIdx, blockIdx) — just arithmetic to find your data

GPU ~132 SM (H100)

GPC — Graphics Processing Cluster Groups of SMs sharing resources

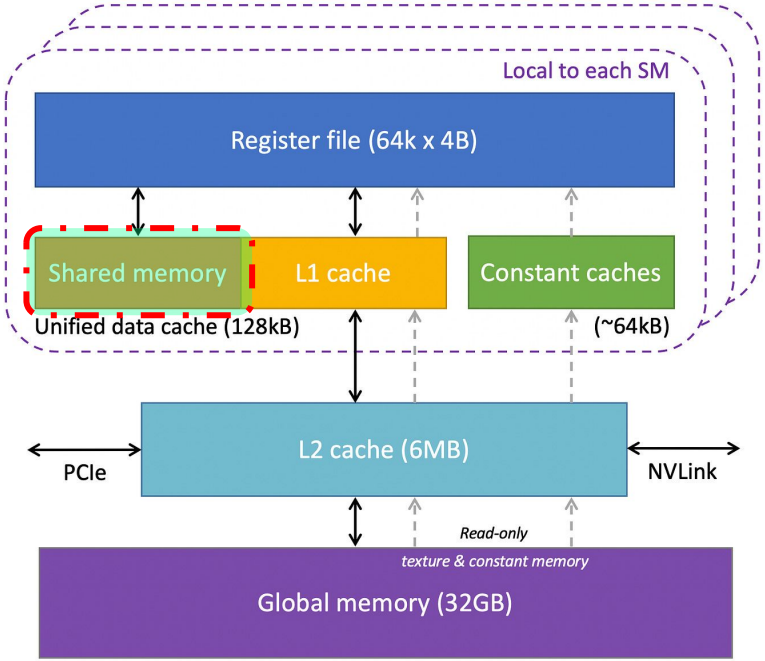
SM — Streaming Multiprocessor 64 CUDA cores + Tensor Cores + 96 KB shared memory

Warp 32 threads executing same instruction (SIMT)

Thread 1 CUDA core, private registers

Level	Problem it solves
Thread	Fine-grained parallelism — one element
Warp	Hardware efficiency — share one task across 32 threads
Block	Shared memory scope — threads in a block can cooperate via fast SRAM
SM	Resource isolation — each SM has its own scheduler, no cross-SM sync
GPU	Massive throughput — 132 SMs running independently

- Общая память (Shared Memory):
  - Программно управляемый кэш, расположенный внутри каждого SM.
  - Быстрая, доступна всем потокам в одном блоке потоков.
  - Используется для обмена данными между потоками внутри блока.
  - Ограниченный размер (например, 64 KB или 96 KB на SM, настраивается в некоторых архитектурах).
  - Работает как scratchpad-память для часто используемых данных.



Регистры

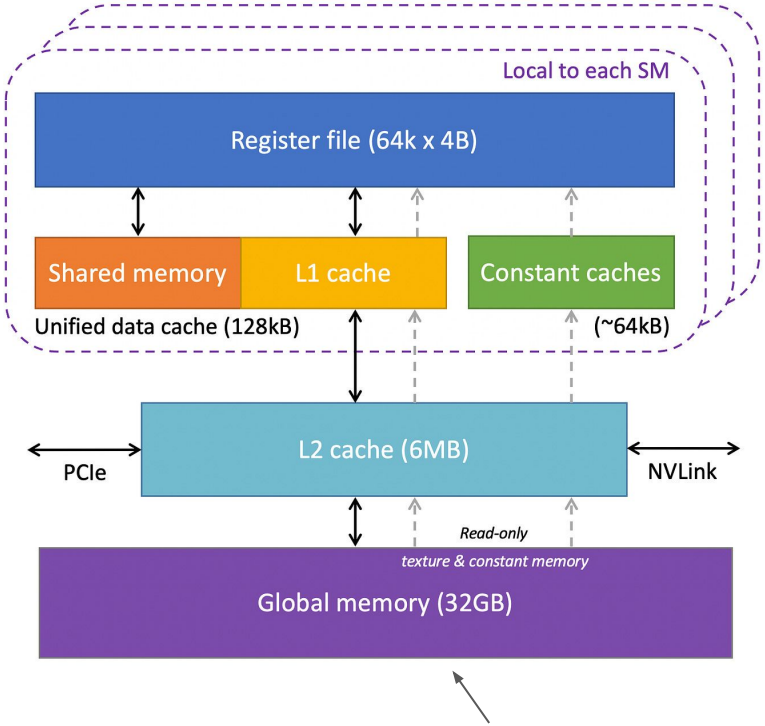
- Самая быстрая память, отдельная для каждого потока.
- Используется для локальных переменных и промежуточных результатов.
- Ограниченное количество (например, 255 регистров/поток в некоторых архитектурах).
- При переполнении данные выгружаются в более медленную память (например, глобальная - DRAM).

Например, модели Nvidia A100 и H100 имеют по 65 536 регистров на один SM.

Глобальная память

- Основная память GPU (DRAM), доступна всем потокам и SM.
- Высокая задержка, но большой объём (например, несколько ГБ).
- Кэшируется кэшем L2 и иногда L1 (зависит от архитектуры).
- Используется для ввода/вывода данных

Например, Nvidia H100 оснащена 80 ГБ памяти с высокой пропускной способностью (HBM) и скоростью передачи данных 3000 ГБ/сек. Из-за значительного расстояния от SM задержка глобальной памяти довольно высока.



Обычно та память про которую мы говорим что ее не хватает.

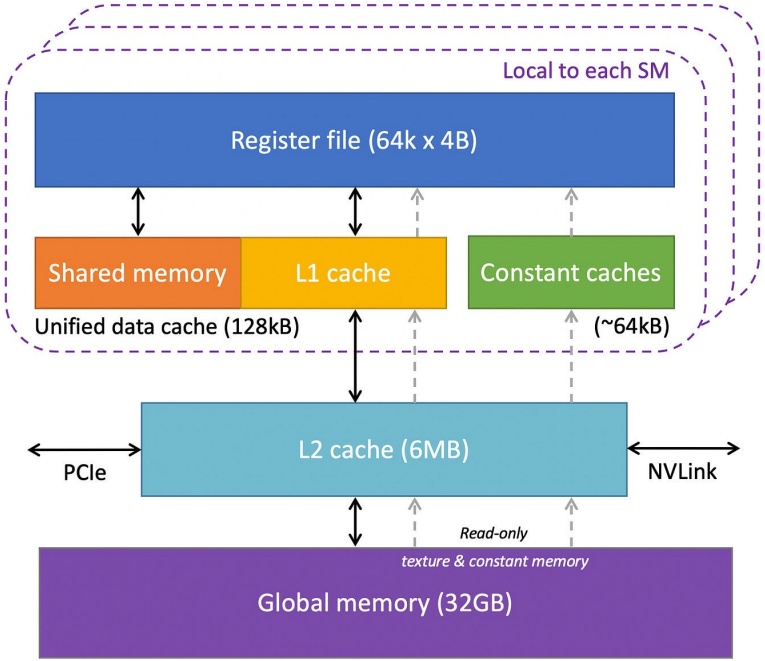
Архитектура GPU

<u>Кэш L1/L2</u>

- <u>Кэш L1</u>: Кэш внутри SM для ускорения доступа к глобальной/локальной памяти.
- <u>Кэш L2</u>: Общий для всех SM, кэширует обращения к глобальной памяти.
- Снижает задержку для часто используемых данных.

<u>Константная память</u>

- Специальная область глобальной памяти только для чтения данных.
- Кэшируется для быстрого доступа при чтении одинаковых значений всеми потоками.
- Используется для констант и параметров ядра.



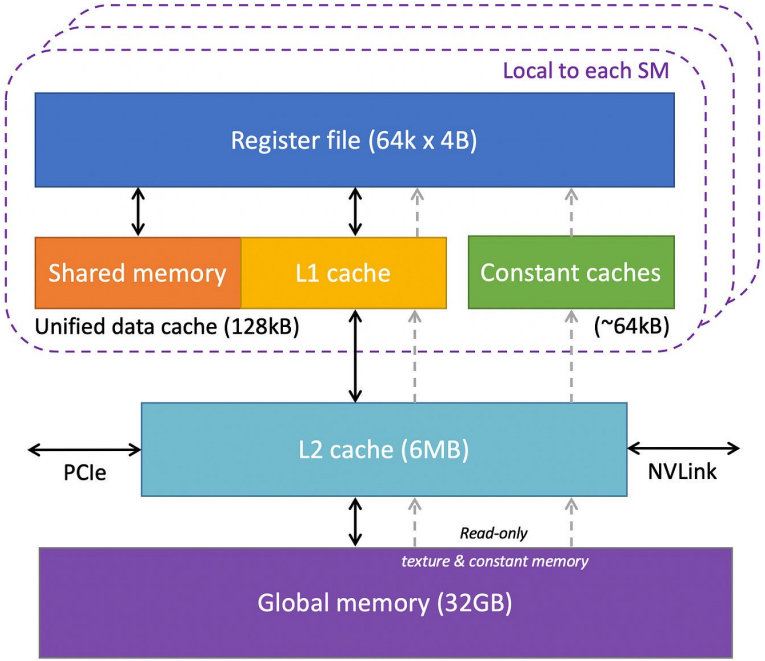
Архитектура GPU

<u>Кэш L1/L2</u>

- <u>Кэш L1</u>: Кэш внутри SM для ускорения доступа к глобальной/локальной памяти.
- <u>Кэш L2</u>: Общий для всех SM, кэширует обращения к глобальной памяти.
- Снижает задержку для часто используемых данных.

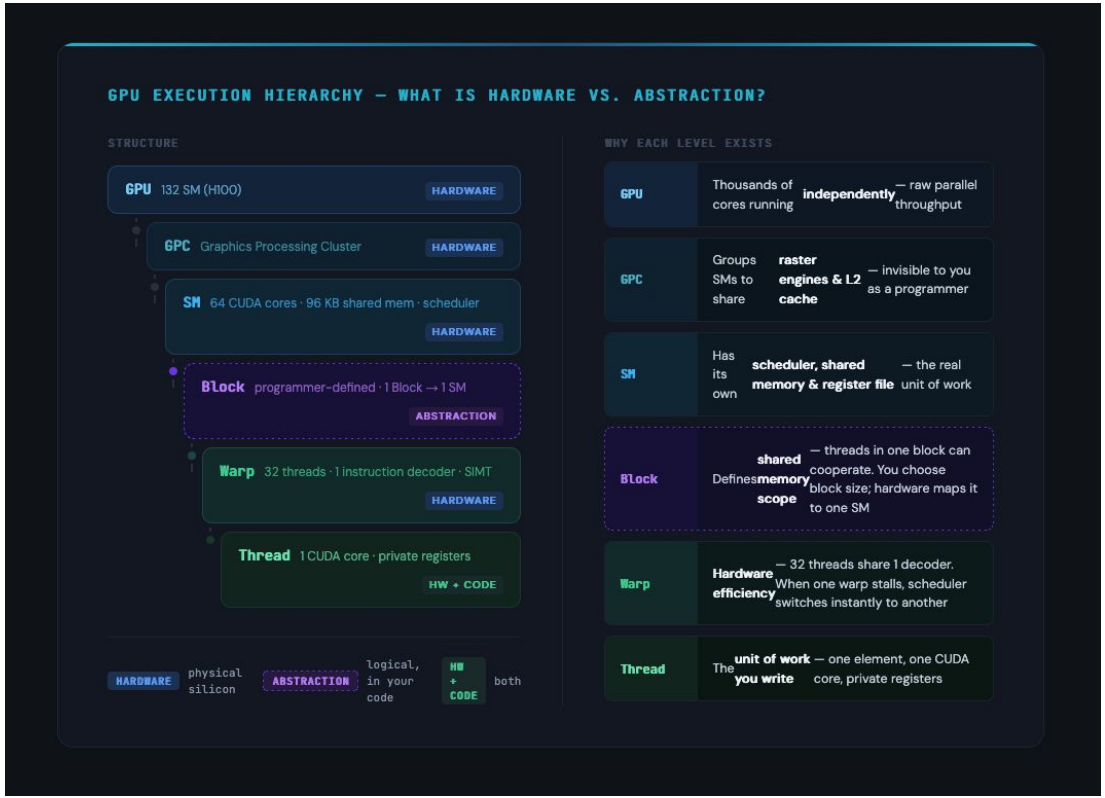
<u>Константная память</u>

- Специальная область глобальной памяти только для чтения данных.
- Кэшируется для быстрого доступа при чтении одинаковых значений всеми потоками.
- Используется для констант и параметров ядра.



What is important:

- Hierarchy in tasks execution
- Hierarchy in memory



4. Arithmetic: Time & Energy Cost MatMul cost analysis

Problem Statement

Matrix Multiplication: C = A × B

- Matrix A: (4096 × 4096) — FP16
- Matrix B: (4096 × 4096) — FP16
- Result C: (4096 × 4096) — FP32
- Hardware: NVIDIA A100 SXM4 80GB

Questions we'll answer:

- How many FLOPs?
- How long does the math take? (compute-bound?)
- How long does data transfer take? (memory-bound?)
- How much energy is consumed?
- FP16 vs FP32: how much difference?

A100 Hardware Specs

- FP16 (Tensor Cores): 312 TFLOPS
- FP32: 19.5 TFLOPS
- HBM bandwidth: 2,000 GB/s
- Memory: 80 GB HBM2e
- TDP (thermal design power): 400 W

Key formulas:

- Time (compute) = FLOPs ÷ Peak FLOPS
- Time (memory) = Bytes ÷ Bandwidth
- Arith. Intensity = FLOPs ÷ Bytes
- Energy = Power × Time

Deriving the FLOPs formula:

For each output element C[i,j]:

$$\Sigma\ A[i,k] \times B[k,j] \quad \text{for } k = 0..K-1$$

K multiplications + K additions = 2K FLOPs

OR with FMA: K multiply-adds = K FLOPs (MACs)

Total output elements: M × N

Total FLOPs = 2 × M × N × K

(Convention: count 1 multiply + 1 add = 2 FLOPs)

Our numbers:

M = N = K = 4096

FLOPs = 2 × 4096 × 4096 × 4096

FLOPs = $2 \times 4096^3 \approx 137.4 \times 10^9$

= 137.4 GFLOPs

Intuition: Scaling

- Double matrix size ⟶ 8× more FLOPs
- MatMul is $O(n^3)$ — very compute-hungry
- A 4096^3 MatMul = 137 billion ops
- **That's why GPUs need Tensor Cores!**

FLOPs vs MACs Confusion

1 MAC = 1 multiply-add = 1 FMA operation

- Convention 1: 1 MAC = 2 FLOPs (multiply + add separately)
- Convention 2: 1 MAC = 1 FLOP (hardware-centric, FMA)

A100 spec uses FLOPs/s counting 1 FMA = 2 FLOPS

Always check which convention a paper uses!

Compute Time (Math-bound)

How fast can the hardware do the math?

Time = FLOPs ÷ Peak Throughput

- FP16 Tensor Cores: 312 TFLOPS

$= 312 \times 10^{12}$ FLOP/s

time compute $= 137.4 \times 10^{9} \div 312 \times 10^{12}$

time compute ≈ 0.44 ms

(at 100% utilization — theoretical lower bound)

FP32 comparison:

- FP32: 19.5 TFLOPS ⟶ time compute ≈ 7.05 ms

FP16 is 16× faster for the same MatMul

Transfer Time (Memory-bound)

How fast can we move data from HBM?

Data to load from global memory:

- A: 4096 × 4096 × 2 bytes = 32 MB
- B: 4096 × 4096 × 2 bytes = 32 MB
- C: 4096 × 4096 × 4 bytes = 64 MB (write)

Total I/O: 128 MB

Time = Bytes ÷ Bandwidth

HBM bandwidth = 2,000 GB/s $= 2 \times 10^{12}$ B/s

Time transfer $= 128 \times 10^{6} \div 2 \times 10^{12}$

Time transfer ≈ 0.064 ms = 64 µs

Verdict: time compute (0.44ms) >> time transfer (0.064ms)

⟶ This MatMul is COMPUTE-BOUND ✓

Arithmetic Intensity (ArI)

ArI = FLOPs ÷ Bytes of I/O

Tells you: which resource is the bottleneck?

Our MatMul:

ArI = $137.4 \times 10^9 \div 128 \times 10^6$

ArI ≈ 1,073 FLOP/byte

Roofline Threshold (A100)

Ridge point = Peak FLOPS ÷ Peak BW

= $312 \times 10^{12} \div 2 \times 10^{12}$

= 156 FLOP/byte

ArI > Ridge point → COMPUTE-BOUND

ArI < Ridge point → MEMORY-BOUND

Our MatMul: 1073 >> 156 → COMPUTE-BOUND ✓

Energy = Power × Time

A100 TDP = 400 W

cuBLAS achieves ~80% utilization in practice:

FP16 real time ≈ 0.44ms ÷ 0.8 ≈ 0.55 ms

FP32 real time ≈ 7.05ms ÷ 0.8 ≈ 8.8 ms

Energy (FP16) = 400 W × 0.55×10^{-3} s

≈ 0.22 Joules

Energy (FP32) = 400 W × 8.8×10^{-3} s

≈ 3.5 Joules

FP16 uses 16× less energy than FP32!

(Same computation, same math, just different precision)

Computing on GPU

Shared memory всего 64 kb

Какого размера квадратная матрица туда влезет в FP32, FP16, Int8?

Computing on GPU

1. For `FP32` (32-bit float, 4 bytes per element)

- **Element size**: 4 bytes.
- **Total elements**:

$$N^2 = \frac{64\ \text{KB}}{4\ \text{bytes}} = \frac{65{,}536}{4} = 16{,}384$$

- **Matrix size**:

$$N = \sqrt{16{,}384} = 128$$

Result: **128×128** matrix.

2. For `FP16` (16-bit float, 2 bytes per element)

- **Element size**: 2 bytes.
- **Total elements**:

$$N^2 = \frac{65{,}536}{2} = 32{,}768$$

- **Matrix size**:

$$N = \sqrt{32{,}768} \approx 181.02 \quad \Rightarrow \quad \text{Floor to } 181$$

Result: **181×181** matrix.

  - Uses $181^2 \times 2 = 65{,}522$ bytes (leaves 14 bytes unused).

3. For `INT8` (8-bit integer, 1 byte per element)

- **Element size**: 1 byte.
- **Total elements**:

$$N^2 = \frac{65{,}536}{1} = 65{,}536$$

- **Matrix size**:

$$N = \sqrt{65{,}536} = 256$$

Result: **256×256** matrix.

Computing on GPU

Data Type	Matrix Size	Memory Used	Unused Bytes
FP32	128 x 128	65,535	0
FP16	181 x 181	65,522	14
INT8	256 x 256	65,535	0

Выполнение операций на GPU

Наши матрицы сильно больше, как
их перемножать?

Tiling

Shared Memory small but fast Global memory large but slow

B (0,0)	B (0,1)	B (0,2)	B (0,3)
B (1,0)	B (1,1)	B (1,2)	B (1,3)
B (2,0)	B (2,1)	B (2,3)	B (3,3)
B (3,0)	B (3,1)	B (3,2)	B (3,3)

A (0,0)	A (0,1)	A (0,2)	A (0,3)
A (1,0)	A (1,1)	A (1,2)	A (1,3)
A (2,0)	A (2,1)	A (2,3)	A (3,3)
A (3,0)	A (3,1)	A (3,2)	A (3,3)

C (0,0)	C (0,1)	C (0,2)	C (0,3)
C (1,0)	C (1,1)	C (1,2)	C (1,3)
C (2,0)	C (2,1)	C (2,3)	C (3,3)
C (3,0)	C (3,1)	C (3,2)	C (3,3)

Thread C [0,0]	Thread C [0,1]	Thread C [1,0]	Thread C [1,1]
A (0,0)	A (0,0)		
A (0,1)	A (0,1)		
A (0,2)	A (0,2)		
A (0,3)	A (0,3)		
B (0,0)	B (0,1)		
B (1,0)	B (1,1)		
B (2,0)	B (2,1)		
B (3,0)	B (3,1)		

Tiling

Shared Memory small but fast Global memory large but slow

Thread [0,0] Thread [0,1]

B (0,0)	B (0,1)	B (0,2)	B (0,3)
B (1,0)	B (1,1)	B (1,2)	B (1,3)
B (2,0)	B (2,1)	B (2,3)	B (3,3)
B (3,0)	B (3,1)	B (3,2)	B (3,3)

A (0,0)	A (0,1)	A (0,2)	A (0,3)
A (1,0)	A (1,1)	A (1,2)	A (1,3)
A (2,0)	A (2,1)	A (2,3)	A (3,3)
A (3,0)	A (3,1)	A (3,2)	A (3,3)

C (0,0)	C (0,1)	C (0,2)	C (0,3)
C (1,0)	C (1,1)	C (1,2)	C (1,3)
C (2,0)	C (2,1)	C (2,3)	C (3,3)
C (3,0)	C (3,1)	C (3,2)	C (3,3)

Thread C [0,0]	Thread C [0,1]	Thread C [1,0]	Thread C [1,1]
A (0,0)	A (0,0)		
A (0,1)	A (0,1)		
A (0,2)	A (0,2)		
A (0,3)	A (0,3)		
B (0,0)	B (0,1)		
B (1,0)	B (1,1)		
B (2,0)	B (2,1)		
B (3,0)	B (3,1)		

Tiling

B (0,0)	B (0,1)	B (0,2)	B (0,3)
B (1,0)	B (1,1)	B (1,2)	B (1,3)
B (2,0)	B (2,1)	B (2,3)	B (3,3)
B (3,0)	B (3,1)	B (3,2)	B (3,3)

- Each thread in a block has unique shared memory
- Global memory can be accessed by all blocks

Tiles fit shared memory

A (0,0)	A (0,1)	A (0,2)	A (0,3)
A (1,0)	A (1,1)	A (1,2)	A (1,3)
A (2,0)	A (2,1)	A (2,3)	A (3,3)
A (3,0)	A (3,1)	A (3,2)	A (3,3)

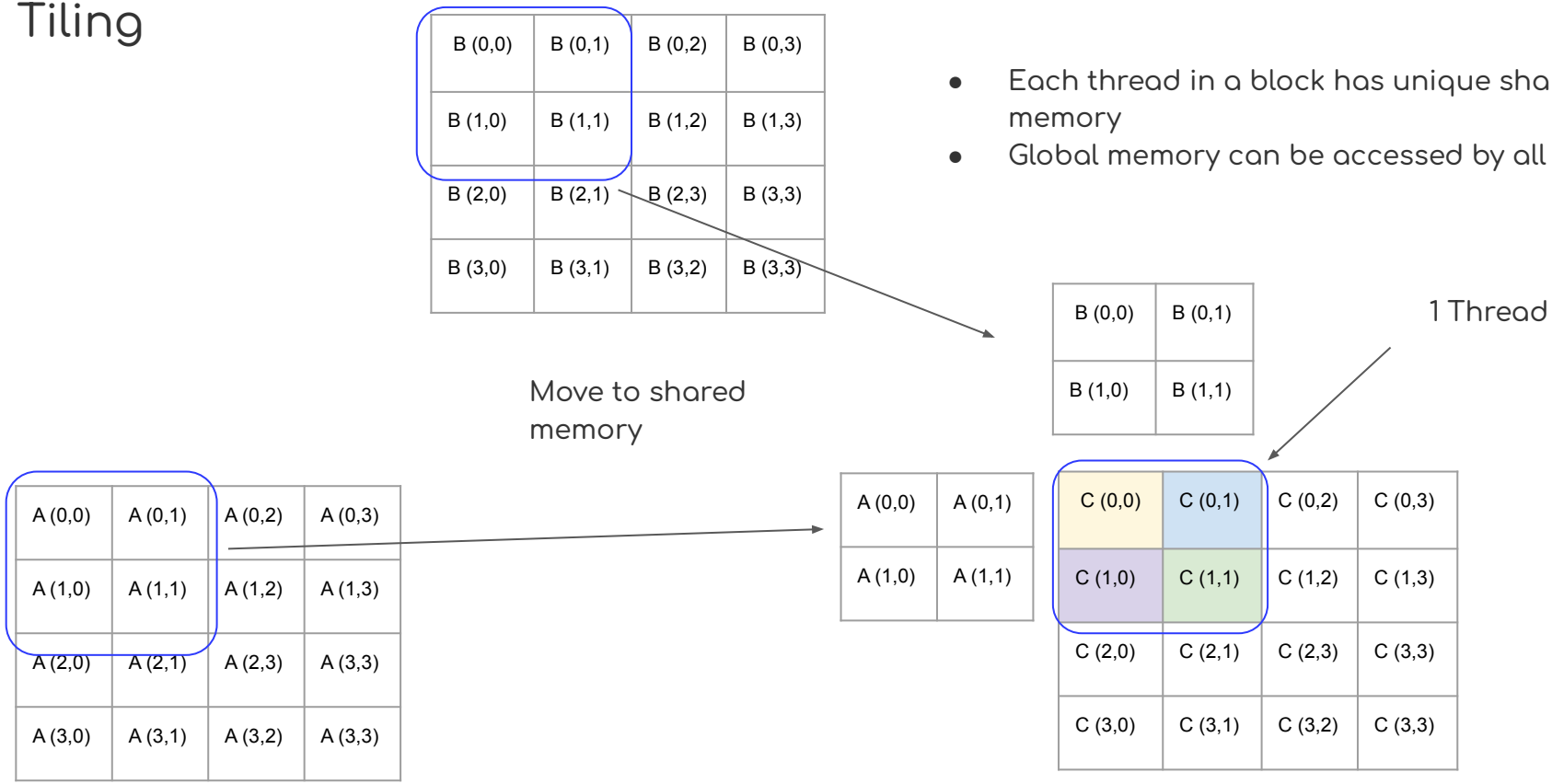
1 Thread

Shared memory

Shared memory

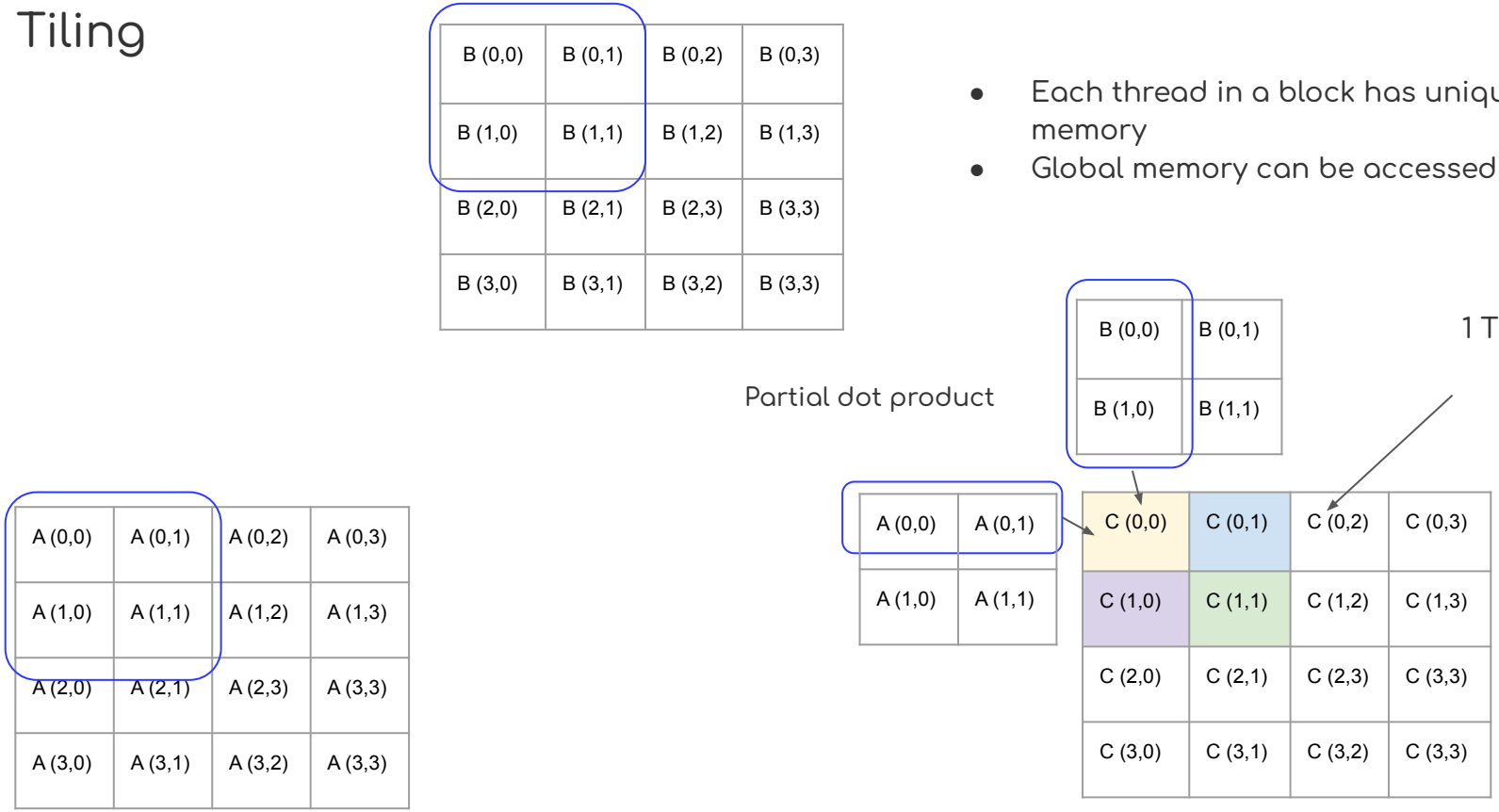
C (0,0)	C (0,1)	C (0,2)	C (0,3)
C (1,0)	C (1,1)	C (1,2)	C (1,3)
C (2,0)	C (2,1)	C (2,3)	C (3,3)
C (3,0)	C (3,1)	C (3,2)	C (3,3)

Tiling



- Each thread in a block has unique shared memory
- Global memory can be accessed by all blocks

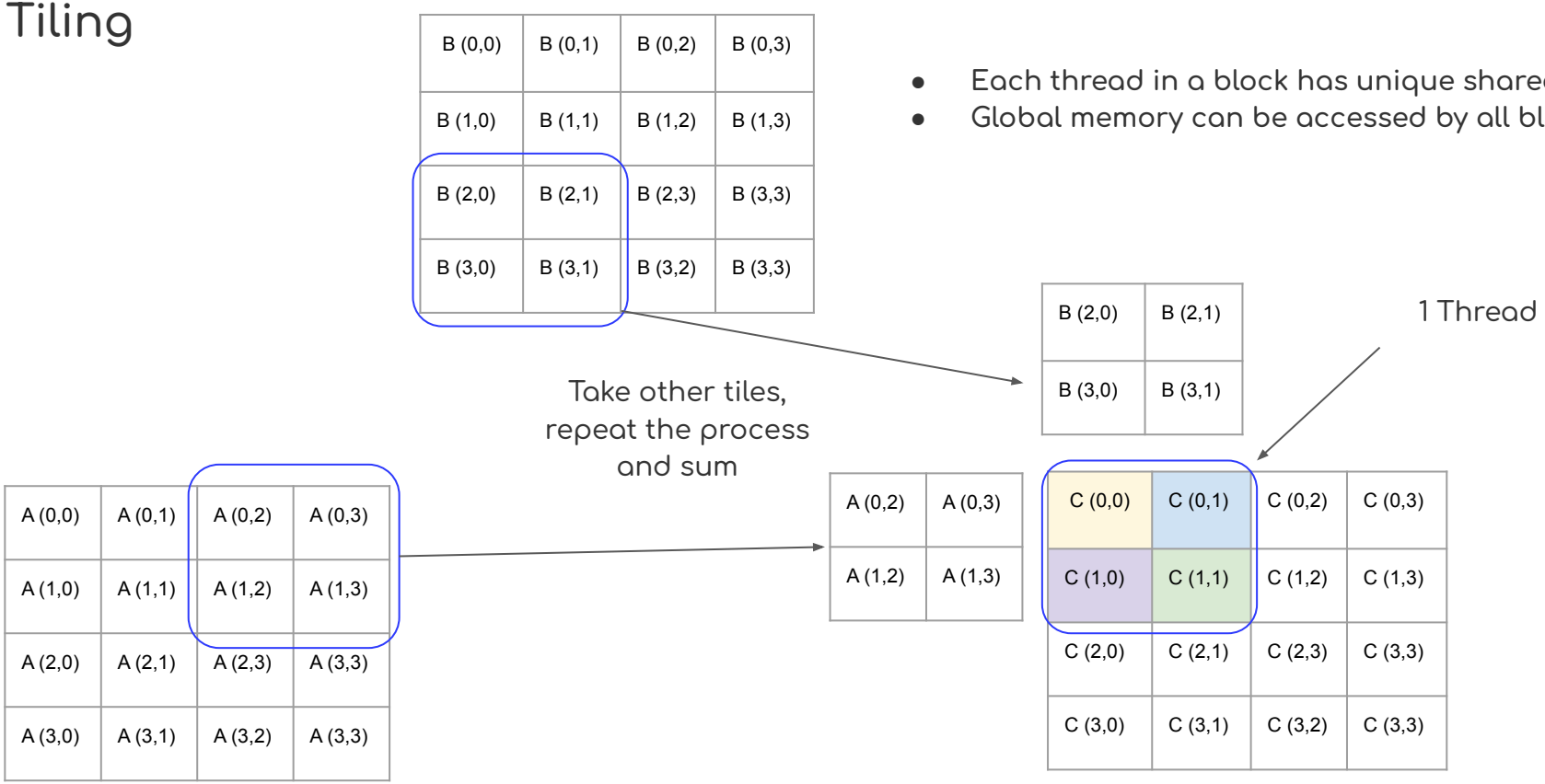
Tiling



- Each thread in a block has unique shared memory
- Global memory can be accessed by all blocks

Tiling

- Each thread in a block has unique shared memory
- Global memory can be accessed by all blocks



Tiling

Shared Memory small but fast Global memory large but slow

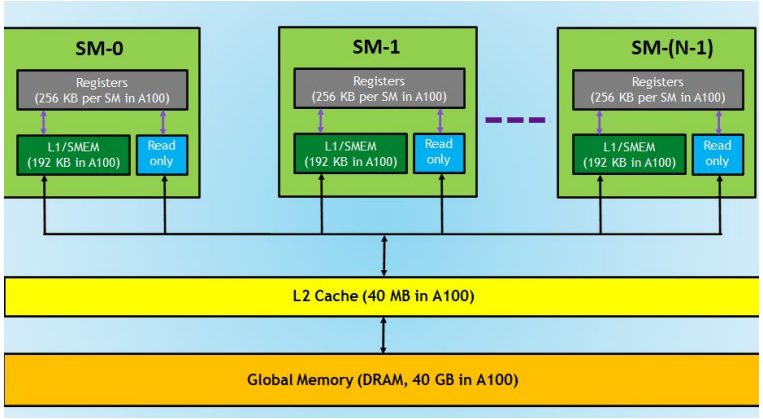
Thread [0,0]	Thread [0,1]	Thread [1,0]	Thread [1,1]
A (0,0)	A (0,0)		
A (0,1)	A (0,1)		
A (0,2)	A (0,2)		
A (0,3)	A (0,3)		
B (0,0)	B (0,1)		
B (1,0)	B (1,1)		
B (2,0)	B (2,1)		
B (3,0)	B (3,1)		

Thread [0,0]	Thread [0,1]	Thread [1,0]	Thread [1,1]
A (0,0)	A (0,1)		
B (0,0)	B (0,1)		
A (0,2)	A (0,3)		
B (2,0)	B (2,1)		

The block as a whole loads less, because shared memory eliminates redundant loads of the same data by different threads.

WARPS, Blocks, etc

Выполнение операций на GPU



Все потоки одного блока назначаются на один и тот же мультипроцессор (SM).

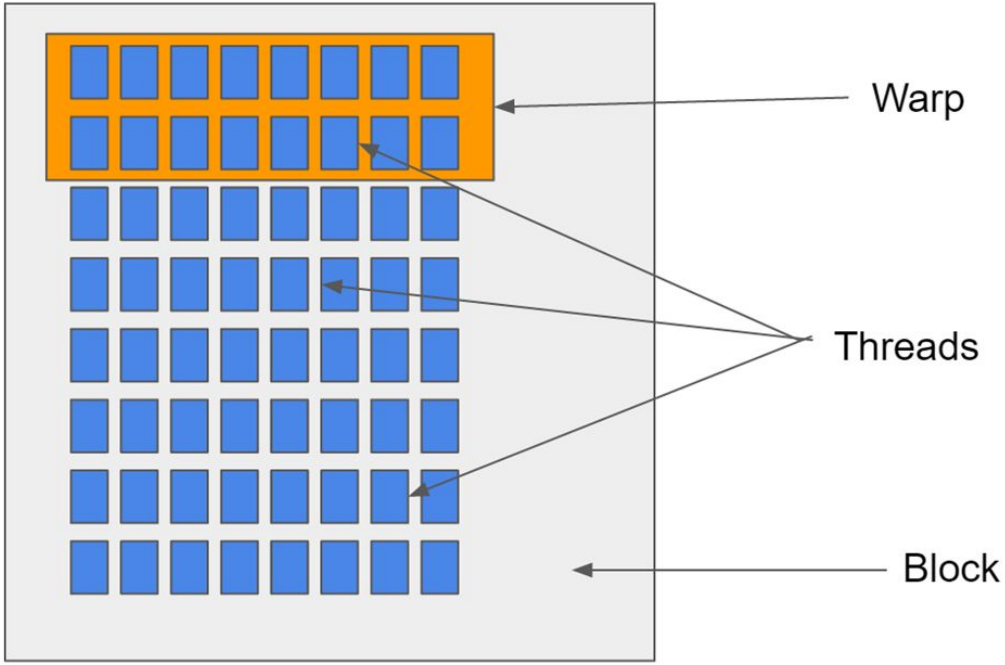
Однако после этого происходит ещё одно деление потоков.

Эти потоки группируются в наборы по 32 потока (warp), которые назначаются для совместного выполнения на наборе ядер, называемом вычислительным блоком.

SM выполняет все потоки в одном warp-е одновременно, извлекая и отправляя одну и ту же инструкцию для всех потоков. Эти потоки затем выполняют эту инструкцию параллельно, но на разных участках данных. В примере со сложением векторов все потоки в warp-е могут выполнять инструкцию сложения, но каждый поток будет работать с разными индексами векторов.

Эта модель выполнения warp-а также называется **SIMT (single instruction multiple threads)** — то есть несколько потоков выполняют одну и ту же инструкцию. Это похоже на SIMD (single instruction multiple data — одиночный поток команд, множественный поток данных) в процессорах.

Выполнение операций на GPU



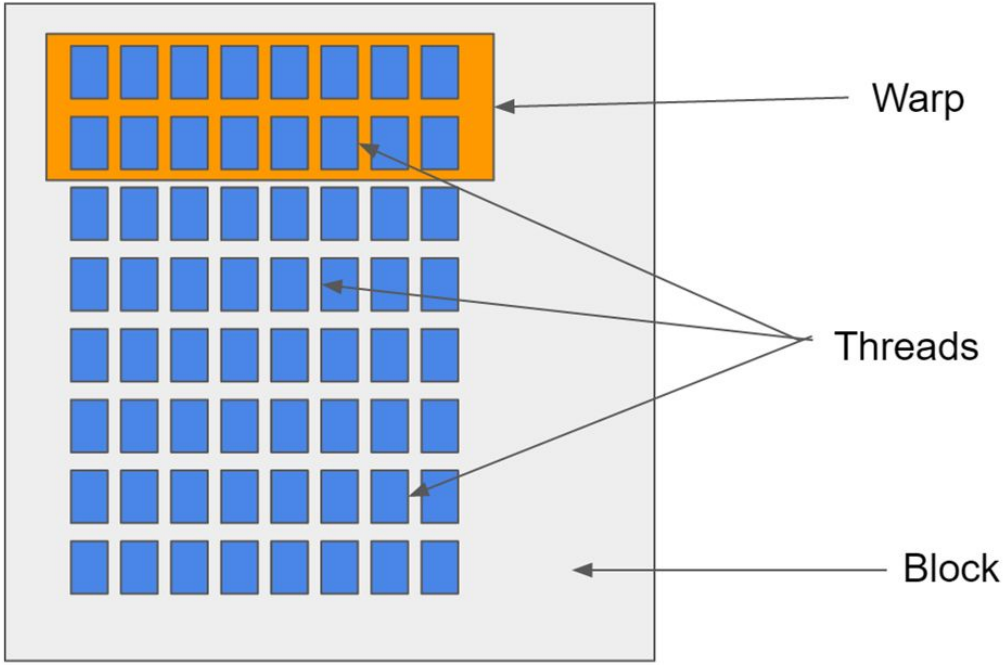
Треды внутри блока физически организованы в варпы (warps).

Треды в одном варпе исполняются вместе и «ждут» друг друга.

На одном SM может одновременно выполняться несколько варпов (порядка 32 - 64).

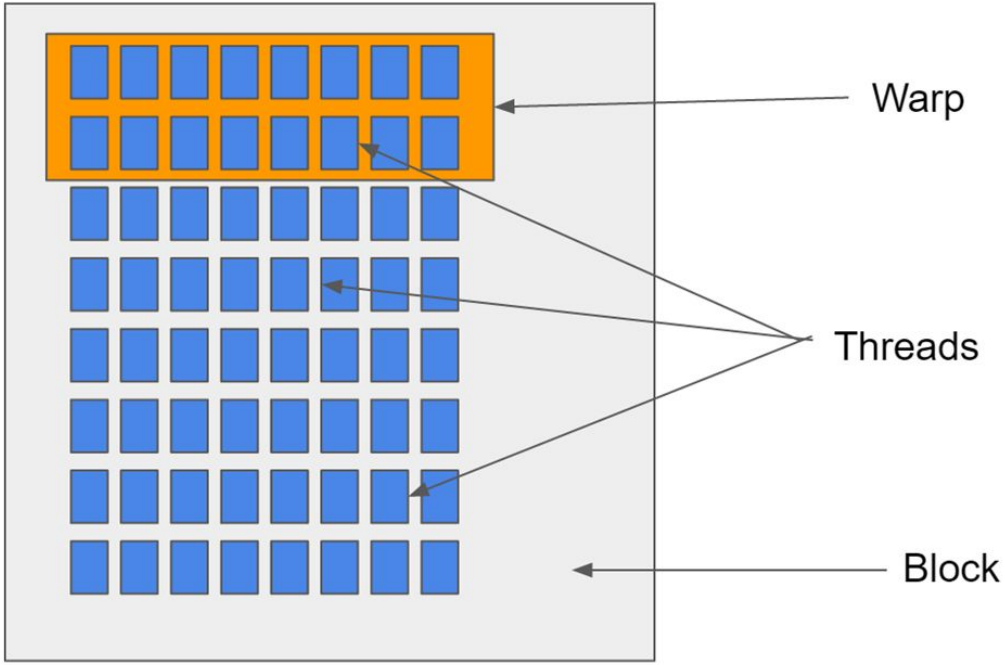
Программист может об этом не задумываться без ущерба для корректности (но не для эффективности!)

Выполнение операций на GPU



А зачем варпы?

Выполнение операций на GPU



А зачем нам Блоки?

Выполнение операций на GPU

	Warp	Block
Инструкция	Одна и таже	
Данные	Warp обрабатывает один tile от матрицы	Разные тайлы
Память	Регистры выделяются на поток внутри варпа	Shared memory распределяется между warps внутри блока

Каждый warp обрабатывает свой тайл матрицы, но инструкции (умножение-накопление) одинаковы для всех.

Выполнение операций на GPU - SUMMARY

1. Запуск ядра и организация потоков

2. Выделение памяти на GPU

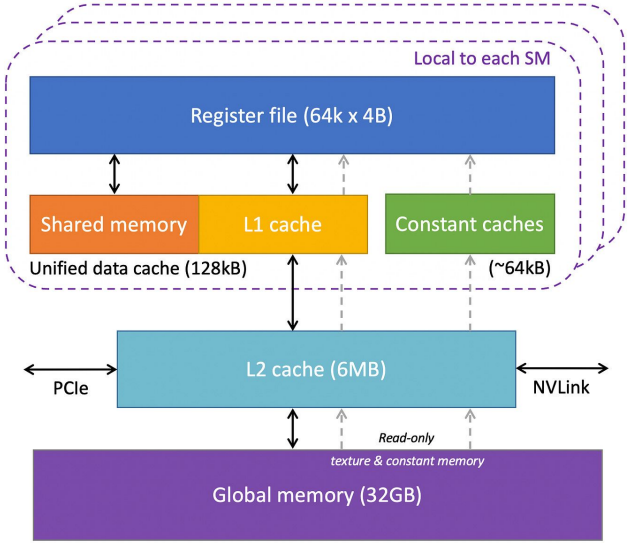
3. Тайлинг: Загрузка тайлов в общую память

4. Синхронизация потоков

5. Вычисление частичных скалярных произведений

6. Повторение тайлинга и накопление

7. Завершение вычислений и запись в глобальную память



Выполнение операций на GPU

```
__global__ void sq_mat_mul_kernel(float* d_A, float* d_B, float*
d_C, int N)
{
    // Identifying the thread mapping
    // Working on C[i,j]
    int i = blockDim.y*blockIdx.y + threadIdx.y;
    int j = blockDim.x*blockIdx.x + threadIdx.x;

    // Check the edge cases
    if (i < N && j < N)
    {
        // Value at C[i,j]
        float value = 0;
        // Loop over elements of the two vectors
        for (int k = 0; k < N; k++)
        {
            // Multiply and add
            value += d_A[i*N+k] * d_B[k*N+j];
        }

        // Assigning calculated value
        d_C[i*N+j] = value;
    }
}
```

- **blockDim:** The dimensions (number of threads) of a thread block.
  - blockDim.x is the number of threads in the x-dimension of the block.
  - blockDim.y is the number of threads in the y-dimension of the block.
- **blockIdx:** The index of the thread block.
  - blockIdx.x is the block's index in the x-dimension of the grid.
  - blockIdx.y is the block's index in the y-dimension of the grid.
- **threadIdx:** The thread's index within its block.
  - threadIdx.x is the thread's index in the x-dimension of the block.
  - threadIdx.y is the thread's index in the y-dimension of the block.

```
C[i][j] = A[i][0]*B[0][j] +
          A[i][1]*B[1][j] +
          A[i][2]*B[2][j] +
          ...
          A[i][N-1]*B[N-1][j]
```

Выполнение операций на GPU

```
__global__ void sq_mat_mul_kernel(float* d_A, float* d_B, float* d_C, int N)
{
    // Identifying the thread mapping
    // Working on C[i,j]
    int i = blockDim.y*blockIdx.y + threadIdx.y;
    int j = blockDim.x*blockIdx.x + threadIdx.x;

    // Check the edge cases
    if (i < N && j < N)
    {
        // Value at C[i,j]
        float value = 0;
        // Loop over elements of the two vectors
        for (int k = 0; k < N; k++)
        {
            // Multiply and add
            value += A[i*N+k] * B[k*N+j];
        }

        // Assigning calculated value
        C[i*N+j] = value;
    }
}
```

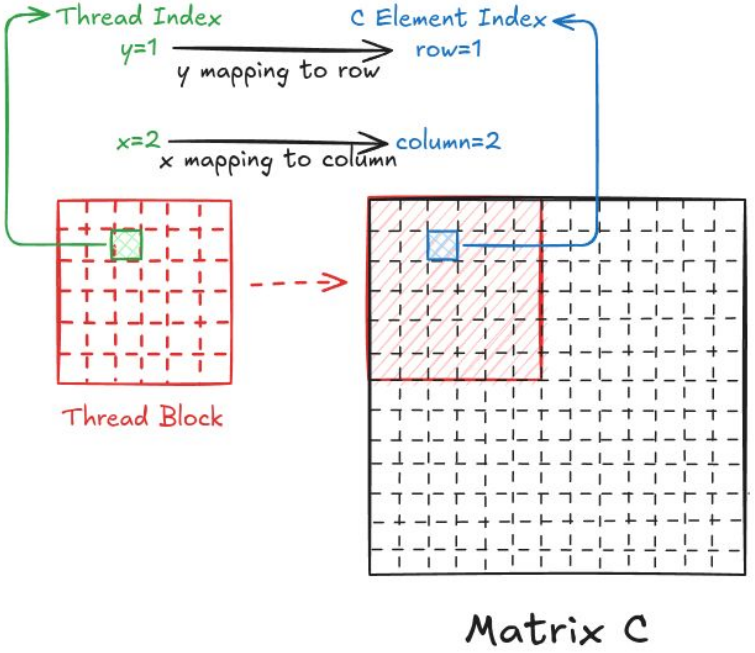
Example:

- N=4, and the grid/block configuration is as follows:
- blockDim.x = 2, blockDim.y = 2 (2x2 threads per block).
- gridDim.x = 2, gridDim.y = 2 (2x2 blocks in the grid).
- This setup creates 4×4=16 threads, one for each element of the 4×4 output matrix d_C.
- Each thread computes one element of d_C based on its global indices (i,j).

source

Выполнение операций на GPU

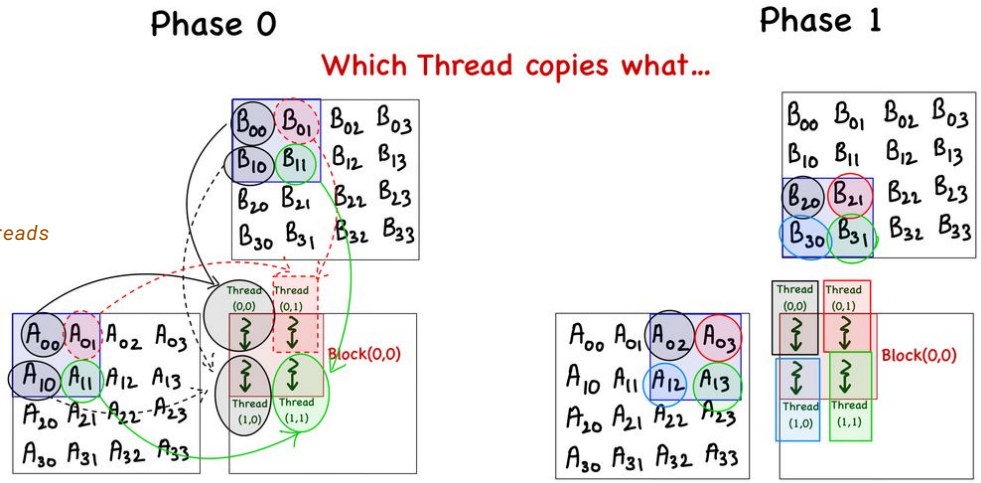
```
__global__ void sq_mat_mul_kernel(float* d_A, float* d_B, float* d_C, int
N)
{
    // Identifying the thread mapping
    // Working on C[i,j]
    int i = blockDim.y*blockIdx.y + threadIdx.y;
    int j = blockDim.x*blockIdx.x + threadIdx.x;

    // Check the edge cases
    if (i < N && j < N)
    {
        // Value at C[i,j]
        float value = 0;
        // Loop over elements of the two vectors
        for (int k = 0; k < N; k++)
        {
            // Multiply and add
            value += A[i*N+k] * B[k*N+j];
        }

        // Assigning calculated value
        C[i*N+j] = value;
    }
}
```



source

Выполнение операций на GPU

```cpp
__global__ void tiled_sq_mat_mul_kernel(float* A, float* B, float* C, int N)
{
    // Ensure that TILE_WIDTH = BLOCK_SIZE
    assert(TILE_WIDTH == blockDim.x);
    assert(TILE_WIDTH == blockDim.y);

    // Ensure N%TILE_WIDTH == 0
    assert(N % TILE_WIDTH == 0);

    // Details regarding this thread
    int by = blockIdx.y;
    int bx = blockIdx.x;

    int ty = threadIdx.y;
    int tx = threadIdx.x;

    // Working on C[i,j]
    int i = TILE_WIDTH*by + ty;
    int j = TILE_WIDTH*bx + tx;

    // Allocating shared memory
    // automatic variables are stored in registers that are private to threads
    __shared__ float sh_A[TILE_WIDTH][TILE_WIDTH];
    __shared__ float sh_B[TILE_WIDTH][TILE_WIDTH];

    // Parallel mat mul
    float value = 0;
    for (int phase = 0; phase < N/TILE_WIDTH; phase++)
    {
        // Load Tiles into shared memory
        sh_A[ty][tx] = A[(i)*N + phase*TILE_WIDTH+tx];
        sh_B[ty][tx] = B[(phase*TILE_WIDTH + ty)*N+j];
        __syncthreads(); // Make sure all threads completed its work

        // Dot product
        for (int k = 0; k < TILE_WIDTH; k++)
            value += sh_A[ty][k] * sh_B[k][tx];
        __syncthreads();
    }
    // Assigning calculated value
    C[i*N+j] = value;
}
```



! two smaller loops will perform faster than one big

Parallel compute on Tiles level

source

Выполнение операций на GPU

```
__global__ void tiled_sq_mat_mul_kernel(float* A, float* B, float* C, int N)
{
    // Ensure that TILE_WIDTH = BLOCK_SIZE
    assert(TILE_WIDTH == blockDim.x);
    assert(TILE_WIDTH == blockDim.y);

    // Ensure N%TILE_WIDTH == 0
    assert(N % TILE_WIDTH == 0);

    // Details regarding this thread
    int by = blockIdx.y;
    int bx = blockIdx.x;

    int ty = threadIdx.y;
    int tx = threadIdx.x;

    // Working on C[i,j]
    int i = TILE_WIDTH*by + ty;
    int j = TILE_WIDTH*bx + tx;

    // Allocating shared memory
    // automatic variables are stored in registers that are private to threads
    __shared__ float sh_A[TILE_WIDTH][TILE_WIDTH];
    __shared__ float sh_B[TILE_WIDTH][TILE_WIDTH];

    // Parallel mat mul
    float value = 0;
    for (int phase = 0; phase < N/TILE_WIDTH; phase++)
    {
        // Load Tiles into shared memory
        sh_A[ty][tx] = A[(i)*N + phase*TILE_WIDTH+tx];
        sh_B[ty][tx] = B[(phase*TILE_WIDTH + ty)*N+j];
        __syncthreads(); // Make sure all threads completed its work

        // Dot product
        for (int k = 0; k < TILE_WIDTH; k++)
            value += sh_A[ty][k] * sh_B[k][tx];
        __syncthreads();
    }
    // Assigning calculated value
    C[i*N+j] = value;
}
```

А где тут Варпы?

source

Выполнение операций на GPU

```
__global__ void tiled_sq_mat_mul_kernel(float* A, float* B, float* C, int N)
{
    // Ensure that TILE_WIDTH = BLOCK_SIZE
    assert(TILE_WIDTH == blockDim.x);
    assert(TILE_WIDTH == blockDim.y);

    // Ensure N%TILE_WIDTH == 0
    assert(N % TILE_WIDTH == 0);

    // Details regarding this thread
    int by = blockIdx.y;
    int bx = blockIdx.x;

    int ty = threadIdx.y;
    int tx = threadIdx.x;

    // Working on C[i,j]
    int i = TILE_WIDTH*by + ty;
    int j = TILE_WIDTH*bx + tx;

    // Allocating shared memory
    // automatic variables are stored in registers that are private to threads
    __shared__ float sh_A[TILE_WIDTH][TILE_WIDTH];
    __shared__ float sh_B[TILE_WIDTH][TILE_WIDTH];

    // Parallel mat mul
    float value = 0;
    for (int phase = 0; phase < N/TILE_WIDTH; phase++)
    {
        // Load Tiles into shared memory
        sh_A[ty][tx] = A[(i)*N + phase*TILE_WIDTH+tx];
        sh_B[ty][tx] = B[(phase*TILE_WIDTH + ty)*N+j];
        __syncthreads(); // Make sure all threads completed its work

        // Dot product
        for (int k = 0; k < TILE_WIDTH; k++)
            value += sh_A[ty][k] * sh_B[k][tx];
        __syncthreads();
    }
    // Assigning calculated value
    C[i*N+j] = value;
}
```

Используются аппаратно

- В коде размер блока задаётся через blockDim.x и blockDim.y.
- Например, если TILE_WIDTH = 32, то блок имеет размер 32x32 = 1024 потока.
- Эти 1024 потока делятся на 32 warp'а (по 32 потока в каждом).

source

Выполнение операций на GPU - SUMMARY

1. Запуск ядра и организация потоков

Запуск CUDA/GPU ядра с **сеткой блоков потоков**, где каждый блок вычисляет подматрицу (тайл) результата.

- Каждому блоку потоков назначается вычисление **тайла** выходной матрицы (например, 16x16 или 32x32 элементов).
- Каждый поток в блоке вычисляет **один элемент** (или небольшой набор) выходного тайла.
- Для сопоставления потоков с индексами матрицы используется **2D-сетка и размеры блоков**

```
void vecAdd(float* A_h, float* B_h, float* C_h, int n) {
    int size = n * sizeof(float);
    float *A_d, *B_d, *C_d;
    cudaMalloc((void **) &A_d, size);
    cudaMalloc((void **) &B_d, size);
    cudaMalloc((void **) &C_d, size);
    cudaMemcpy(A_d, A_h, size, cudaMemcpyHostToDevice);
    cudaMemcpy(B_d, B_h, size, cudaMemcpyHostToDevice);

    // Launch ceil(n/256) blocks of 256 threads each
    vecAddKernel<<<ceil(n/256.0), 256>>>(A_d, B_d, C_d, n);

    cudaMemcpy(C_h, C_d, size, cudaMemcpyDeviceToHost);
    cudaFree(A_d);
    cudaFree(B_d);
    cudaFree(C_d);
}
```

A_h, B_h, C_h are float vectors holding data stored in the host (CPU) memory.

n is the size of each vector

A_d, B_d, C_d are vectors for holding data in device (GPU) memory.

We need to allocate memory for these using the cudaMalloc API of CUDA, which allocates memory on the GPU.

And, then we need to copy A_h, and B_h to A_d, and B_d respectively. The cudaMemcpy function copies data between host and device memories.

This executes the function vecAddKernel on the GPU with A_d, B_d, C_d, and n as parameters. The stuff between <<< and >>> is the kernel grid configuration: number of blocks and number of threads per block.

Finally, we copy the result from C_d to C_h, i.e. transfer it from device (GPU) memory to host (CPU) memory.
We also free the memory allocated on the GPU.

```
__global__
void vecAddKernel(float* A, float* B, float* C, int n) {
    int i = threadIdx.x + blockDim.x * blockIdx.x;

    if (i < n) {
        C[i] = A[i] + B[i];
    }
}
```

__global__ declares a function as a kernel function in CUDA, which can be executed on the GPU.

In CUDA, threadIdx and blockDim are built-in variables used to manage the organization of threads. Threads are grouped into blocks, and each block contains a specific number of threads. For example, you might have three blocks, each consisting of 256 threads.

threadIdx.x: This variable provides the unique ID of a thread within its block, starting from 0. For example, the first thread in a block has threadIdx.x equal to 0.

blockDim.x: This variable represents the number of threads in a block and, indirectly, the block's ID. It tells you how many threads are in each block. For instance, if you have three blocks, and you're running code in the first block, blockDim.x would typically be 256, and blockIdx.x would be 0.

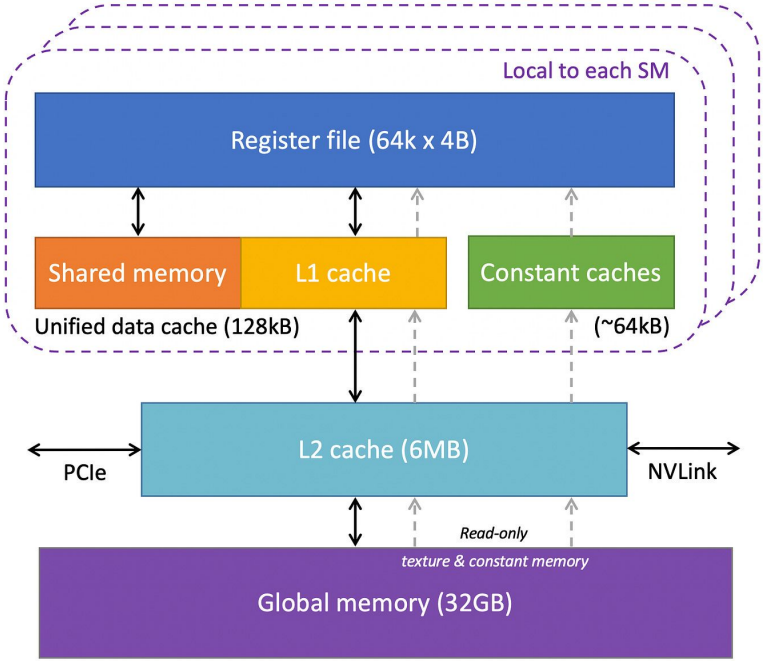
The computation of i serves a crucial purpose.
It allows each thread to calculate a unique index that can be used to access different elements of vectors or arrays.

Выполнение операций на GPU - SUMMARY

2. Выделение памяти на GPU

Копирование входных матриц **A** и **B** с CPU (хоста) в глобальную память GPU (устройства).

- **Глобальная память**: Хранит матрицы **A** и **B** для доступа всех потоков/SM.
- **Общая память**: Зарезервирована для каждого блока, чтобы кэшировать тайлы **A** и **B**.



Выполнение операций на GPU - SUMMARY

3. Тайлинг: Загрузка тайлов в общую память

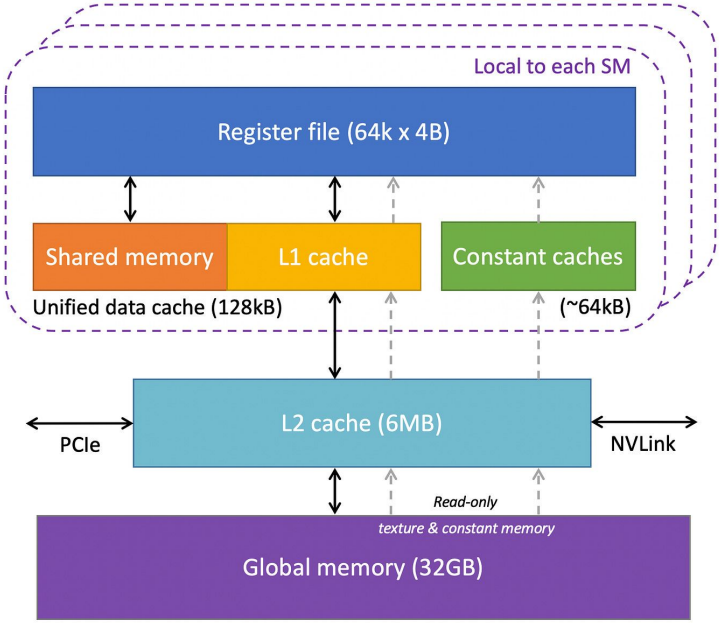
Каждый блок потоков загружает **тайлы** матриц **A** и **B** из глобальной памяти в общую память.

Тайлинг: Разбивает матрицы на меньшие тайлы (например, 16x16), которые помещаются в общую память.

Совместная загрузка: Потоки в блоке работают вместе, чтобы загрузить тайлы:

- Каждый поток загружает **один элемент** из глобальной памяти в общую память.
- Используется **коалесцированный доступ к памяти** (соседние потоки обращаются к соседним адресам глобальной памяти) для повышения эффективности.

Общая память: Тайлы **A** и **B** кэшируются здесь для быстрого повторного доступа.

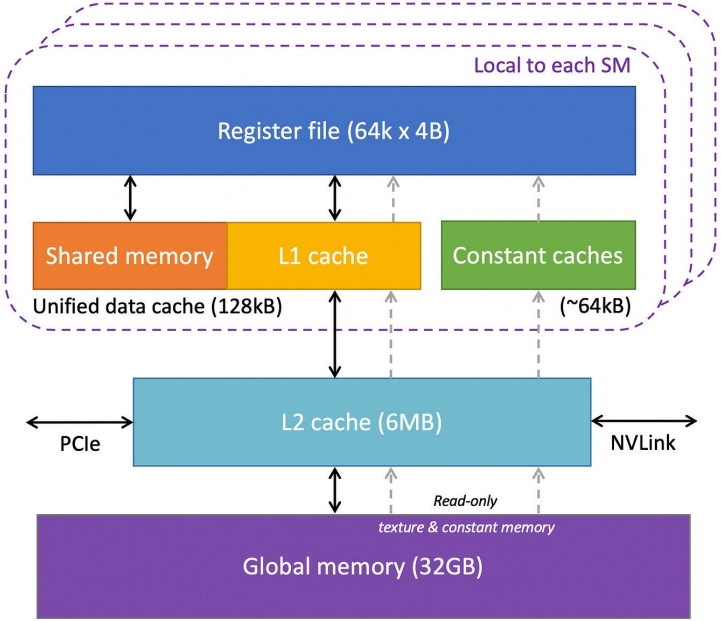


Выполнение операций на GPU - SUMMARY

5. Вычисление частичных скалярных произведений

Каждый поток вычисляет частичное скалярное произведение для своего назначенного элемента выходной матрицы.

- **Цикл по тайлам**: Для каждой пары тайлов (из **A** и **B**) вычислите скалярное произведение.
- **Регистры**: Хранят промежуточные результаты (например, `sum += A_tile[row][k] * B_tile[k][col]`)
- **Эффективность на уровне warps**: Warps (32 потока) выполняют инструкции синхронно (SIMD), скрывая задержки за счёт переключения контекста.



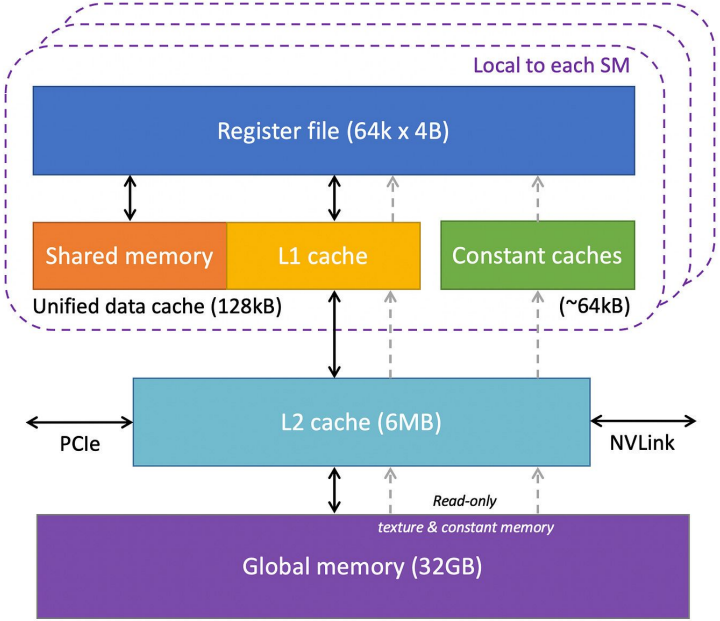
Выполнение операций на GPU - SUMMARY

6. Повторение тайлинга и накопление

Загрузка следующей пары тайлов в общую память, и вычисление их вклада в скалярное произведение, накопление результатов.

- Цикл продолжается, пока все тайлы строки (для **A**) и столбца (для **B**) не будут обработаны.

- Тайлы в общей памяти повторно используются, что сокращает доступ к глобальной памяти.

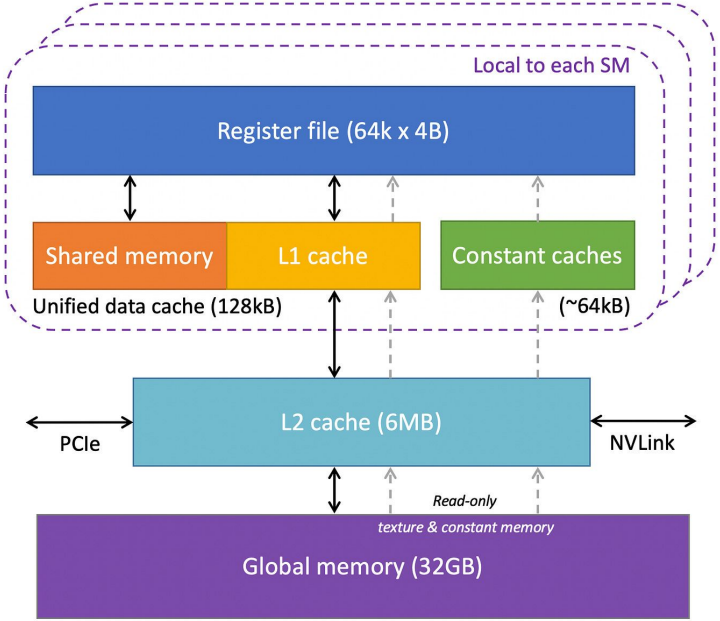


Выполнение операций на GPU - SUMMARY

7. Завершение вычислений и запись в глобальную память

Запись вычисленных значений (хранящееся в регистрах) в выходную матрицу **C** в глобальной памяти.

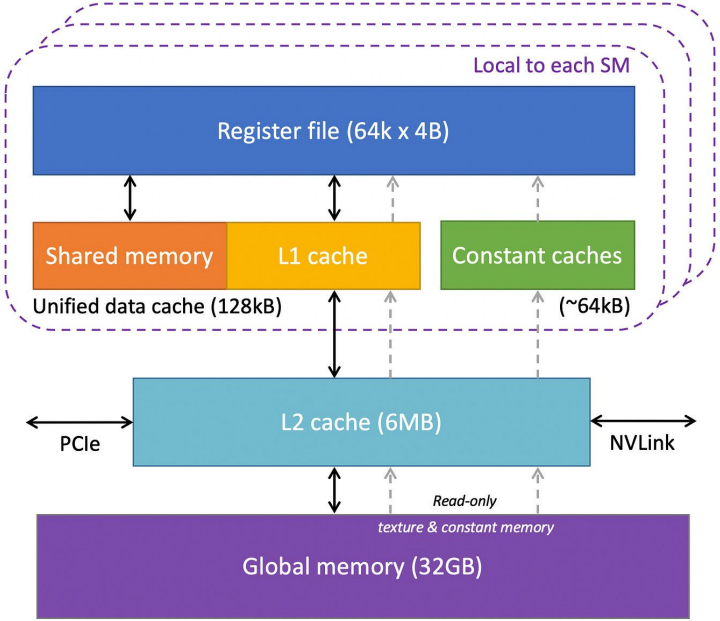
- Каждый поток записывает свой результат в правильную позицию в **C** через глобальную память.
- **Коалесцированные записи**: Обеспечивает что соседние потоки записывают данные в соседние адреса памяти для повышения эффективности.



Выполнение операций на GPU - SUMMARY

Иерархия памяти

1. **Глобальная память**: Исходные данные (A, B) и конечный результат (C).
2. **Общая память**: Кэшированные тайлы A и B для быстрого доступа.
3. **Кэш L1/L2**: Кэширование часто используемых данных из глобальной памяти (например, повторно используемых элементов матриц).
4. **Регистры**: Хранение промежуточных сумм, приватных для потоков.



Вопросы?